

Exact Diffusion Inversion via Bidirectional Integration Approximation

Guoqiang Zhang¹, J.P. Lewis^{2,3}, and W. Bastiaan Kleijn³

¹ University of Exeter, United Kingdom, g.z.zhang@exeter.ac.uk

² NVIDIA Research, USA, jpl@nvidia.com

³ Victoria Univ. of Wellington, New New Zealand, bastiaan.kleijn@vuw.ac.nz

Abstract. Recently, various methods have been proposed to address the inconsistency issue of DDIM inversion to enable image editing, such as EDICT [40] and Null-text inversion [24]. However, the above methods introduce considerable computational overhead. In this paper, we propose a new technique, *bidirectional integration approximation* (BDIA), to perform exact diffusion inversion with negligible computational overhead. We consider a family of second order integration algorithms obtained by averaging forward and backward DDIM steps. The resulting approach estimates the next diffusion state as a linear combination of the estimated Gaussian noise at the current step and the previous and current diffusion states. This allows for exact backward computation of previous state given the current and next ones, leading to exact diffusion inversion. We perform a convergence analysis for BDIA-DDIM that includes the analysis for DDIM as a special case. It is demonstrated with experiments that BDIA-DDIM is effective for (round-trip) prompt-driven image editing. Our experiments further show that BDIA-DDIM produces markedly better image sampling quality than DDIM and EDICT for text-to-image generation and conventional image sampling.⁴

1 Introduction

As one type of generative models [1, 5, 10, 11, 31], diffusion probabilistic models (DPMs) have made significant progress in recent years. Following the work of [34], various learning and/or sampling strategies have been proposed to improve the performance of DPMs, which include, for example, denoising diffusion probabilistic models (DDPMs) [12], denoising diffusion implicit models (DDIMs) [35], improved DDIMs [7, 25], latent diffusion models (LDMs) [27], score matching with Langevin dynamics (SMLD) [36–38], analytic-DPMs [3, 4], optimized denoising schedules [6, 19, 20], guided diffusion strategies [17, 26], and classifier-free guided diffusion [13]. It is worth noting that DDIM can be interpreted as a first-order ordinary differential equation (ODE) solver. As an extension of DDIM, various high-order ODE solvers have been proposed, such as EDM [16], DEIS [44], PLMS and PNDM [21], DPM-Solvers++ [23], and IIA-EDM and IIA-DDIM [42].

⁴ BDIA can also be applied to improve the performance of other ODE solvers in addition to DDIM. In particular, it is found that applying BDIA to the EDM sampling procedure produces consistently better performance over four pre-trained models (see Alg. 4 and Table 3 in Appendix C).



Fig. 1: Comparison of images generated by using DDIM, BDIA-DDIM and EDICT with 10 timesteps over StableDiffusion V2. EDICT needs to perform 20 NFEs, which is two times of the NFEs needed in BDIA-DDIM. See Appendix I for an explanation why EDICT does not perform well for one-way noise-to-image generation. See Table 1 for FID and CLIP evaluation, and Figs 7-8 for more image comparisons. Please also see Appendix K in the supplementary material for additional text-based and ControlNet-based image-editing results.

In recent years, image-editing via diffusion models has attracted increasing attention in both academia and industry. One important operation for editing a real image is to first perform forward process on the image to obtain the final noise representation and then perform a backward process with embedded editing to generate the desired image [28, 30]. DDIM inversion has been widely used to perform the above forward and backward processes [32]. A major issue with DDIM inversion is that the intermediate diffusion states in the forward and backward processes may be inconsistent due to the inherent approximations (see Subsection 4.1). This issue becomes significant when utilizing classifier-free guidance in text-to-image editing [32]. The newly generated images are often perceptually far away from the original ones, which is undesirable.

Recently, two methods have been proposed to address the inconsistency issue of DDIM inversion. Specifically, the work of [24] proposed a technique named *null-text inversion* to push the diffusion states of the backward process to be optimally close to those of the forward process via iterative optimization. The null-text inputs to the score neural network are treated as free variables in the optimization procedure. In [40], the authors proposed the EDICT technique to enforce exact DDIM inversion. Their basic idea is to introduce an auxiliary diffusion state and then perform alternating updates on the primal and auxiliary diffusion states, which is inspired by the flow generative framework [8, 9, 18]. One drawback of EDICT is that the number of neural function evaluations (NFEs) has to be doubled in comparison to DDIM inversion (See Subsection 4.2). Another related line of research work is DDPM inversion (see [14]).

In this paper, we propose a new technique to enforce exact DDIM inversion with negligible computational overhead, reducing the number of NFEs required in EDICT by half. Suppose we are in a position to estimate the next diffusion state z_{i-1} at timestep t_i by utilizing the two most recent states z_i and z_{i+1} . With the estimated Gaussian noise $\hat{\epsilon}(z_i, i)$, we perform the DDIM update procedure

twice for approximating the ODE integration over the next time-slot $[t_i, t_{i-1}]$ in the forward manner and the previous time-slot $[t_i, t_{i+1}]$ in the backward manner. Note that in this paper *forward* refers to the denoising direction. The DDIM for the previous time-slot is employed to refine the integration approximation made earlier when computing $\mathbf{z}_i$. As a result, the expression for $\mathbf{z}_{i-1}$ becomes a linear combination of $(\mathbf{z}_{i+1}, \mathbf{z}_i, \hat{\epsilon}(\mathbf{z}_i, i))$, and naturally facilitates exact diffusion inversion⁵. We refer to the above technique as *bidirectional integration approximation* (BDIA). We provide a convergence analysis for BDIA-DDIM that follows an approach similar to that used for the linear multi-step methods in the literature of numerical integration [2]. Experiments demonstrate that BDIA-DDIM produces promising results on both image sampling and image editing (Figs. 1, 2, 3, 5, 6, 7, 8, 9, 10, 11, Table 1, 2).

2 Related Work

From a high-level point of view, BDIA-DDIM can be viewed as a variant of linear multistep methods in the literature of numerical integration [2]. A typical linear multistep method employs multiple historical states and gradients in estimation of the next state when solving an ODE. One main difference between BDIA-DDIM and the linear multistep methods is that BDIA-DDIM performs bidirectional semi-linear integration approximation (see [22] for viewing the DDIM update expression as a semi-linear integration approximation). As a result, the convergence analysis for BDIA-DDIM is slightly different from that for linear multi-step methods as presented in Theorem 6.6 of [2].

BDIA-DDIM is also related to the existing work of time-reversible ODE solvers. For instance, Verlet integration is a time-reversible method for solving 2nd-order ODEs [39]. Leapfrog integration is another time-reversible method also developed for solving 2nd-order ODEs [33].

The recent work [41] proposed a new diffusion sampling method, named Restart sampling, that adds Gaussian noise in a backward manner from time to time. The main difference is that BDIA solves an ODE without adding any Gaussian noise in the sampling process whereas Restart alternates the ODE solver step and the insertion of Gaussian noise.

3 Preliminary

Forward and reverse diffusion processes: Suppose the data sample $\mathbf{x} \in \mathbb{R}^d$ follows a data distribution $p_{data}(\mathbf{x})$ with a bounded variance. A forward diffusion process progressively adds Gaussian noise to the data samples $\mathbf{x}$ to obtain $\mathbf{z}_t$ as t increases from 0 until T . The conditional distribution of $\mathbf{z}_t$ given $\mathbf{x}$ can be represented as

$$q_{t|0}(\mathbf{z}_t|\mathbf{x}) = \mathcal{N}(\mathbf{z}_t|\alpha_t\mathbf{x}, \sigma_t^2\mathbf{I}) \quad \mathbf{z}_t = \alpha_t\mathbf{x} + \sigma_t\epsilon, \quad (1)$$

⁵ In principle, one can utilize more previous diffusion states when computing $\mathbf{z}_{i-1}$ to improve accuracy (see A.2 for a discussion). However, this would incur additional memory overhead, which may be undesirable.

where α_t and σ_t are assumed to be differentiable functions of t with bounded derivatives. It is assumed that the three functions α_t , σ_t , and $\lambda_t = \frac{\sigma_t}{\alpha_t}$, are non-increasing, non-decreasing, and strictly increasing (see [22]) as t increases, respectively. We use $q(\mathbf{z}_t; \alpha_t, \sigma_t)$ to denote the marginal distribution of $\mathbf{z}_t$. The samples of the distribution $q(\mathbf{z}_T; \alpha_T, \sigma_T)$ should be practically indistinguishable from pure Gaussian noise if $\sigma_T \gg \alpha_T$.

The reverse process of a diffusion model firstly draws a sample $\mathbf{z}_T$ from $\mathcal{N}(\mathbf{0}, \sigma_T^2 \mathbf{I})$, and then progressively denoises it to obtain a sequence of diffusion states $\{\mathbf{z}_{t_i} \sim p(\mathbf{z}; \alpha_{t_i}, \sigma_{t_i})\}_{i=0}^N$, where we use the notation $p(\cdot)$ to indicate that the reverse sample distribution might not be identical to $q(\cdot)$ because of practical approximations. It is expected that the final sample $\mathbf{z}_{t_0}$ is roughly distributed according to $p_{data}(\mathbf{x})$, i.e., $p_{data}(\mathbf{x}) \approx p(\mathbf{z}_{t_0}; \alpha_{t_0}, \sigma_{t_0})$ where $t_0 = 0$.

ODE formulation: In [38], Song et al. present a so-called *probability flow* ODE which shares the same marginal distributions as $\mathbf{z}_t$ in (1). Specifically, with the formulation (1) for a forward diffusion process, its reverse ODE form can be represented as [22, 42]

$$d\mathbf{z} = \overbrace{\left[\frac{d \log \alpha_t}{dt} \mathbf{z}_t - 1/2 \left(\frac{d\sigma_t^2}{dt} - 2 \frac{d \log \alpha_t}{dt} \sigma_t^2 \right) \nabla_{\mathbf{z}} \log q(\mathbf{z}_t; \alpha_t, \sigma_t) \right]}^{d(\mathbf{z}_t, t)} dt, \quad (2)$$

where $d(\mathbf{z}_t, t)$ denotes the gradient vector at time t . $\nabla_{\mathbf{z}} \log q(\mathbf{z}; \alpha_t, \sigma_t)$ in (2) is the score function [15] pointing towards higher density of data samples at the given noise level (α_t, σ_t) . One nice property of the score function is that it does not depend on the generally intractable normalization constant of the underlying density function $q(\mathbf{z}; \alpha_t, \sigma_t)$.

As t decreases, the probability flow ODE (2) continuously reduces the noise level of the data samples in the reverse process. In the ideal scenario where no approximations are introduced in (2), the sample distribution $p(\mathbf{z}; \alpha_t, \sigma_t)$ approaches the original data distribution $p_{data}(\mathbf{x})$ as t goes from T to 0. As a result, the sampling process of a diffusion model boils down to solving the ODE form (2), where randomness is only introduced in the initial sample at time T . This has opened up the research opportunity of exploiting different ODE solvers in diffusion-based sampling processes.

Denoising score matching: To be able to utilize (2) for sampling, one needs to specify a particular form of the score function $\nabla_{\mathbf{z}} \log q(\mathbf{z}; \alpha_t, \sigma_t)$. One common approach is to train a noise estimator $\hat{\epsilon}_{\theta}$ by minimizing the expected L_2 error for samples drawn from q_{data} (see [12, 35, 38]):

$$\mathbb{E}_{\mathbf{x} \sim p_{data}} \mathbb{E}_{\epsilon \sim \mathcal{N}(\mathbf{0}, \sigma_t^2 \mathbf{I})} \|\hat{\epsilon}_{\theta}(\alpha_t \mathbf{x} + \sigma_t \epsilon, t) - \epsilon\|_2^2, \quad (3)$$

where (α_t, σ_t) are from the forward process (1). The common practice in diffusion models is to utilize a neural network of U-Net architecture [29] to represent the noise estimator $\hat{\epsilon}_{\theta}$. With (3), the score function can then be represented in terms of $\hat{\epsilon}_{\theta}(\mathbf{z}_t; t)$ as (see also (229) of [16])

$$\nabla_{\mathbf{z}} \log q(\mathbf{z}_t; \alpha_t, \sigma_t) = \frac{-(\mathbf{z}_t - \alpha_t \mathbf{x})}{\sigma_t^2} = -\hat{\epsilon}_{\theta}(\mathbf{z}_t; t) / \sigma_t. \quad (4)$$

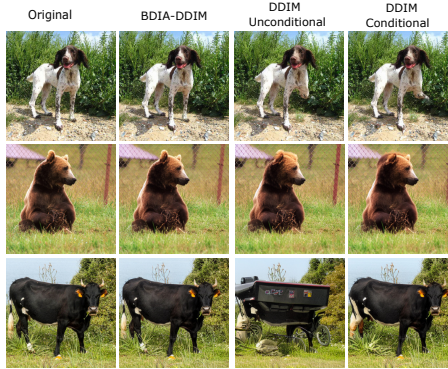


Fig. 2: BDIA-DDIM produces perceptually better image reconstruction (via round-trip) than DDIM. EDICT produces very similar results as BDIA-DDIM with two times of NFEs needed by BDIA-DDIM.

4 Bidirectional Integration Approximation (BDIA)

In this section, we first review DDIM inversion and EDICT as an extension of DDIM inversion. We then present our BDIA technique to enable exact diffusion inversion. As an example, Fig. 2 shows the performance of BDIA-DDIM for image reconstruction in comparison to that of DDIM. Convergence analysis for BDIA-DDIM is performed in the end.

4.1 Review of DDIM inversion

We first consider the update expression of DDIM for sampling, which is in fact a first-order solver for the ODE formulation (2) (see [22, 44]), given by

$$\mathbf{z}_{i-1} = \alpha_{i-1} \left(\frac{\mathbf{z}_i - \sigma_i \hat{\epsilon}_{\theta}(\mathbf{z}_i, i)}{\alpha_i} \right) + \sigma_{i-1} \hat{\epsilon}_{\theta}(\mathbf{z}_i, i) \quad (5)$$

$$= a_i \mathbf{z}_i + b_i \hat{\epsilon}_{\theta}(\mathbf{z}_i, i) \approx \mathbf{z}_i + \int_{t_i}^{t_{i-1}} \mathbf{d}(\mathbf{z}_{\tau}, \tau) d\tau, \quad (6)$$

where $a_i = \alpha_{i-1}/\alpha_i$ and $b_i = \sigma_{i-1} - \sigma_i \alpha_{i-1}/\alpha_i$. It is clear from (5)-(6) that the integration $\int_{t_i}^{t_{i-1}} \mathbf{d}(\mathbf{z}_{\tau}, \tau) d\tau$ is approximated by the forward DDIM update. That is, only the diffusion state $\mathbf{z}_i$ at the starting timestep t_i is used in the integration approximation.

To perform DDIM inversion, $\mathbf{z}_i$ can be approximated in terms of $\mathbf{z}_{i-1}$ as

$$\mathbf{z}_i = \alpha_i \left(\frac{\mathbf{z}_{i-1} - \sigma_{i-1} \hat{\epsilon}_{\theta}(\mathbf{z}_{i-1}, i)}{\alpha_{i-1}} \right) + \sigma_i \hat{\epsilon}_{\theta}(\mathbf{z}_{i-1}, i) \quad (7)$$

$$\approx \alpha_i \left(\frac{\mathbf{z}_{i-1} - \sigma_{i-1} \hat{\epsilon}_{\theta}(\mathbf{z}_{i-1}, i)}{\alpha_{i-1}} \right) + \sigma_i \hat{\epsilon}_{\theta}(\mathbf{z}_{i-1}, i), \quad (8)$$

where $\mathbf{z}_i$ in the RHS of (7) is replaced with $\mathbf{z}_{i-1}$ to facilitate explicit computation. This naturally introduces approximation errors, leading to inconsistency of the diffusion states between the forward and backward processes.

4.2 Review of EDICT for exact diffusion inversion

Inspired by the flow generative framework [18], the recent work [40] proposed EDICT to enforce exact diffusion inversion. The basic idea is to introduce an auxiliary diffusion state $\mathbf{y}_i$ to be coupled with $\mathbf{z}_i$ at every timestep i . The next pair of diffusion states $(\mathbf{z}_{i-1}, \mathbf{y}_{i-1})$ is computed in an alternating fashion as

$$\mathbf{z}_i^{\text{inter}} = a_i \mathbf{z}_i + b_i \hat{\epsilon}_\theta(\mathbf{y}_i, i) \quad (9)$$

$$\mathbf{y}_i^{\text{inter}} = a_i \mathbf{y}_i + b_i \hat{\epsilon}_\theta(\mathbf{z}_i^{\text{inter}}, i) \quad (10)$$

$$\mathbf{z}_{i-1} = p \mathbf{z}_i^{\text{inter}} + (1-p) \mathbf{y}_i^{\text{inter}} \quad (11)$$

$$\mathbf{y}_{i-1} = p \mathbf{y}_i^{\text{inter}} + (1-p) \mathbf{z}_{i-1}, \quad (12)$$

where $p \in [0, 1]$ is the weighting factor and the pair $(\mathbf{z}_i^{\text{inter}}, \mathbf{y}_i^{\text{inter}})$ represents the intermediate diffusion states. According to [40], the two averaging operations (11)-(12) are introduced to make the update procedure stable.

Due to the alternating update formalism in (9)-(12), the computation can be inverted to obtain $(\mathbf{z}_i, \mathbf{y}_i)$ in terms of $(\mathbf{z}_{i-1}, \mathbf{y}_{i-1})$ as

$$\mathbf{y}_i^{\text{inter}} = (\mathbf{y}_{i-1} - (1-p)\mathbf{z}_{i-1})/p \quad (13)$$

$$\mathbf{z}_i^{\text{inter}} = (\mathbf{z}_{i-1} - (1-p)\mathbf{y}_{i-1}^{\text{inter}})/p \quad (14)$$

$$\mathbf{y}_i = (\mathbf{y}_i^{\text{inter}} - b_i \hat{\epsilon}_\theta(\mathbf{z}_i^{\text{inter}}, i))/a_i \quad (15)$$

$$\mathbf{z}_i = (\mathbf{z}_i^{\text{inter}} - b_i \hat{\epsilon}_\theta(\mathbf{y}_i, i))/a_i \quad (16)$$

Unlike (7)-(8), the inversion of (9)-(12) does not involve any approximation, thus enabling exact diffusion inversion.

However, it is clear from the above equations that the NFEs needed in EDICT is two times the NFEs required for DDIM. This makes the method computationally expensive. It is highly desirable to reduce the NFE in EDICT while retaining exact diffusion inversion. We provide such a method in the next subsection.

4.3 BDIA-DDIM for exact diffusion inversion

Reformulation of DDIM update expression: In this section, we present our new technique BDIA to assist DDIM in achieving exact diffusion inversion. To do so, we first reformulate the update expression for $\mathbf{z}_{i-1}$ in (6) in terms of all the historical diffusion states $\{\mathbf{z}_j\}_{j=N}^i$ as

$$\mathbf{z}_{i-1} = \mathbf{z}_N + \sum_{j=N}^i \Delta(t_j \rightarrow t_{j-1} | \mathbf{z}_j) \quad (17)$$

$$\approx \mathbf{z}_N + \sum_{j=N}^i \int_{t_j}^{t_{j-1}} d(\mathbf{z}_\tau, \tau) d\tau, \quad (18)$$

where we use $\Delta(t_j \rightarrow t_{j-1} | \mathbf{z}_j)$ to denote approximation of the integration $\int_{t_j}^{t_{j-1}} d(\mathbf{z}_\tau, \tau) d\tau$ via the forward DDIM step, given by

$$\Delta(t_j \rightarrow t_{j-1} | \mathbf{z}_j) = \mathbf{z}_{j-1} - \mathbf{z}_j = a_j \mathbf{z}_j + b_j \hat{\epsilon}_\theta(\mathbf{z}_j, j) - \mathbf{z}_j. \quad (19)$$

Algorithm 1 BDIA-DDIM for improving sampling quality

```

1: Input: number of time steps  $N$ ,  $\gamma \in [0, 1]$ 
2: Sample  $\mathbf{z}_N \sim \mathcal{N}(\mathbf{0}, \sigma_{t_N}^2 \mathbf{I})$ 
3:  $\mathbf{z}_{N-1} = a_N \mathbf{z}_N + b_N \hat{\mathbf{e}}_\theta(\mathbf{z}_N, N)$ 
4: for  $i \in \{N-1, \dots, 1\}$  do
5:    $\mathbf{z}_{i-1} = \gamma(\mathbf{z}_{i+1} - \mathbf{z}_i) - \gamma \left( \frac{\mathbf{z}_i}{a_{i+1}} - \frac{b_{i+1}}{a_{i+1}} \hat{\mathbf{e}}_\theta(\mathbf{z}_i, i) - \mathbf{z}_i \right) + \left[ a_i \mathbf{z}_i + b_i \hat{\mathbf{e}}_\theta(\mathbf{z}_i, i) \right]$ 
6: end for
7: Output:  $\mathbf{z}_0$ 

```

Extending forward DDIM by average of forward and backward DDIM:

We argue that, in principle, each integration $\int_{t_j}^{t_{j-1}} \mathbf{d}(\mathbf{z}_\tau, \tau) d\tau$ in (18) can be alternatively approximated by taking the average of both the forward and backward DDIM updates, expressed as

$$\int_{t_j}^{t_{j-1}} \mathbf{d}(\mathbf{z}_\tau, \tau) d\tau \approx (1 - \phi_{i,j}) \Delta(t_j \rightarrow t_{j-1} | \mathbf{z}_j) - \phi_{i,j} \Delta(t_{j-1} \rightarrow t_j | \mathbf{z}_{j-1}), \quad (20)$$

where the scalar $\phi_{i,j} \in [0, 1]$, and the notation $\Delta(t_{j-1} \rightarrow t_j | \mathbf{z}_{j-1})$ denotes the backward DDIM step from t_{j-1} to t_j . The minus sign in front of $\Delta(t_{j-1} \rightarrow t_j | \mathbf{z}_{j-1})$ is due to integration over reverse time. The update expression for the backward DDIM step can be represented as

$$\Delta(t_{j-1} \rightarrow t_j | \mathbf{z}_{j-1}) = \frac{\mathbf{z}_{j-1}}{a_j} - \frac{b_j}{a_j} \hat{\mathbf{e}}_\theta(\mathbf{z}_{j-1}, j-1) - \mathbf{z}_{j-1}, \quad (21)$$

where the expressions for (a_j, b_j) are similar to (a_i, b_i) in (6). Note that in practice, we first need to perform a forward DDIM step over $[t_j, t_{j-1}]$ to obtain $\mathbf{z}_{j-1}$, and then we are able to perform the backward DDIM step computing $\Delta(t_{j-1} \rightarrow t_j | \mathbf{z}_{j-1})$.

We expect the averaged integration approximation (20) to be more accurate than the forward integration approximation alone. In particular, the averaged integration approximation should have a similar effect as the updates (see (57)-(58) in the appendix) of the improved Euler method which reduces the integration approximation error of the Euler method by utilizing the gradients at the two end points of a time-interval.

Bidirectional integration approximation (BDIA): We now present our new BDIA technique. Our primary goal is to develop an update expression for each $\mathbf{z}_{i-1}$ as a linear combination of $(\mathbf{z}_{i+1}, \mathbf{z}_i, \hat{\mathbf{e}}_\theta(\mathbf{z}_i, i))$. As will be explained in the following, the summation of the integrations $\sum_{j=N}^i \int_{t_j}^{t_{j-1}} \mathbf{d}(\mathbf{z}_\tau, \tau) d\tau$ for $\mathbf{z}_{i-1}$ will involve both forward and backward DDIM updates as in (20).

We first draw insights from the updates for $\mathbf{z}_{N-1}$ and $\mathbf{z}_{N-2}$, and then propose a general update expression for $\mathbf{z}_i$, $i < N-1$. Suppose we are at the initial time step t_N with state $\mathbf{z}_N$. Then the next state $\mathbf{z}_{N-1}$ is computed by applying the forward DDIM (see (19)):

$$\mathbf{z}_{N-1} = a_N \mathbf{z}_N + b_N \hat{\mathbf{e}}_\theta(\mathbf{z}_N, N) = \mathbf{z}_N + \Delta(t_N \rightarrow t_{N-1} | \mathbf{z}_N). \quad (22)$$

Next we consider computing $\mathbf{z}_{N-2}$. With $\mathbf{z}_{N-1}$, we are able to compute $\Delta(t_{N-1} \rightarrow t_N | \mathbf{z}_{N-1})$ over the previous time-slot $[t_{N-1}, t_N]$ and $\Delta(t_{N-1} \rightarrow t_{N-2} | \mathbf{z}_{N-1})$ over the next time-slot $[t_{N-1}, t_{N-2}]$. When computing $\mathbf{z}_{N-2}$, the integration $\int_{t_N}^{t_{N-1}} \mathbf{d}(\mathbf{z}_\tau, \tau) d\tau$ can be approximated by averaging both $-\Delta(t_{N-1} \rightarrow t_N | \mathbf{z}_{N-1})$ and $(\mathbf{z}_{N-1} - \mathbf{z}_N) = \Delta(t_N \rightarrow t_{N-1} | \mathbf{z}_N)$. Other diffusion states $\mathbf{z}_i$, $i < N - 2$, can be computed similarly as $\mathbf{z}_{N-2}$. Based on the above analysis, we propose to compute $\mathbf{z}_{i-1}$ for $i \leq N - 1$ as:

$$\mathbf{z}_{i-1} = \mathbf{z}_{i+1} - \underbrace{(1 - \gamma)(\mathbf{z}_{i+1} - \mathbf{z}_i) - \gamma \Delta(t_i \rightarrow t_{i+1} | \mathbf{z}_i)}_{\approx \int_{t_{i+1}}^{t_i} \mathbf{d}(\mathbf{z}_\tau, \tau) d\tau} + \underbrace{\Delta(t_i \rightarrow t_{i-1} | \mathbf{z}_i)}_{\approx \int_{t_i}^{t_{i-1}} \mathbf{d}(\mathbf{z}_\tau, \tau) d\tau} \quad (23)$$

$$= \gamma \mathbf{z}_{i+1} - \gamma \left[\frac{\mathbf{z}_i}{a_{i+1}} - \frac{b_{i+1}}{a_{i+1}} \hat{\epsilon}_\theta(\mathbf{z}_i, i) \right] + \left[a_i \mathbf{z}_i + b_i \hat{\epsilon}_\theta(\mathbf{z}_i, i) \right], \quad (24)$$

where $\gamma \in [0, 1]$. For the special case that $\gamma = 1$, (24) takes a simple form

$$\mathbf{z}_{i-1} = \mathbf{z}_{i+1} - \Delta(t_i \rightarrow t_{i+1} | \mathbf{z}_i) + \Delta(t_i \rightarrow t_{i-1} | \mathbf{z}_i). \quad (25)$$

It is clear from (23) that in the computation of $\mathbf{z}_{i-1}$, the integration approximation $\int_{t_{i+1}}^{t_i} \mathbf{d}(\mathbf{z}_\tau, \tau) d\tau$ for the previous time-slot $[t_{i+1}, t_i]$ is approximated by taking average of the backward DDIM update $-\Delta(t_i \rightarrow t_{i+1} | \mathbf{z}_i)$ and the integration approximation $\mathbf{z}_i - \mathbf{z}_{i+1}$ made earlier for the same time-slot. When $\gamma = 1$, the integration $\int_{t_{i+1}}^{t_i} \mathbf{d}(\mathbf{z}_\tau, \tau) d\tau$ is simply approximated by the backward DDIM update $-\Delta(t_i \rightarrow t_{i+1} | \mathbf{z}_i)$. On the other hand, when $\gamma = 0$, (24) reduces to the regular DDIM update.

In principle, the computational complexities of DDIM and BDIA-DDIM sampling procedures should be roughly the same. BDIA-DDIM only incurs additional memory consumption to store the latest diffusion state $\mathbf{z}_{i+1}$.

Next we derive the explicit expression for $\mathbf{z}_i$ in terms of all the historical forward and backward DDIM updates, which is summarized in the following:

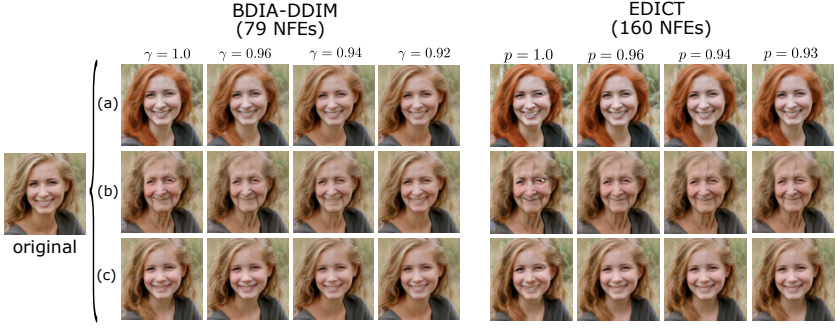
Proposition 1. *Let $\mathbf{z}_{N-1}$ and $\{\mathbf{z}_i | i \leq N - 2\}$ be computed by following (22) and (24) sequentially. Then for each timestep $i \leq N - 2$, $\mathbf{z}_i$ can be represented in the form of*

$$\mathbf{z}_i = \mathbf{z}_N + \sum_{j=i+2}^N \left[\frac{1 - (-\gamma)^{j-i}}{1 + \gamma} \Delta(t_j \rightarrow t_{j-1} | \mathbf{z}_j) - \frac{\gamma + (-\gamma)^{j-i}}{1 + \gamma} \Delta(t_{j-1} \rightarrow t_j | \mathbf{z}_{j-1}) \right] \quad (26)$$

$$+ \Delta(t_{i+1} \rightarrow t_i | \mathbf{z}_{i+1}). \quad (27)$$

Proof. See Appendix A for proof.

We now investigate (26)-(27). It is immediate from (26) that for each time-slot $[t_j, t_{j-1}]$, the summation of the two coefficients $(1 - (-\gamma)^{j-i})/(1 + \gamma)$ and $(\gamma + (-\gamma)^{j-i})/(1 + \gamma)$ is equal to 1. This implies that the usage of BDIA indeed leads to the averaged forward and backward DDIM updates per time-slot as we planned



original: "a blonde woman" (a): "a woman with red hair" (b): "an old woman" (c): "a girl"

Fig. 3: Demonstration of the impact of γ and p values in (round-trip) image-editing performance of BDIA-DDIM and EDICT, respectively. The number of timesteps was set to 40 in both methods. The plots for BDIA-DDIM indicate that as γ decreases from 1 to 0.92, the edited images tend to be visually closer to the original image. That is, the γ parameter in BDIA-DDIM provides one more degree of freedom to allow for flexible image-editing than DDIM. The figure also indicates that the performance of EDICT with $p = 1$ has noticeable distortions while BDIA-DDIM with $\gamma = 1$ produces better image quality, and at the same time, only consumes half of the NFEs needed by EDICT. The recommended setup for p in EDICT is $p = 0.93$. See Appendix L for additional experiments by utilizing Null-text inversion.

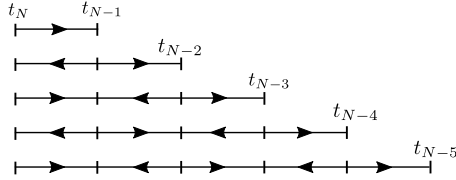


Fig. 4: Schematic illustration of the BDIA-DDIM integration formulation for $\gamma = 1$. Right and left arrows denote forward and backward updates, respectively.

earlier in (20). For the special case of $\gamma = 1$, the update expression (40) becomes much simpler (see Corollary 1 for the simplified expression). Fig. 4 demonstrates how the entire integration $\int_{t_N}^{t_i} \mathbf{d}(\mathbf{z}_\tau, \tau) d\tau$ for different $\mathbf{z}_i$ is approximated when $\gamma = 1$. It can be seen from the figure that the directions of the integration approximation for neighbouring time-slots are always opposite.

Next, we briefly discuss the averaging operations in EDICT and the proposed BDIA technique. It is seen from (11)-(12) that EDICT performs averaging via the parameter p over the two paired diffusion states $\mathbf{z}$ and $\mathbf{y}$, both of which are computed via forward DDIM updates (see Appendix I for an analysis of the update inconsistency in EDICT). On the other hand, BDIA proposes to average via the parameter γ over the DDIM forward and backward updates. See Fig. 3 for how the averaging operations affect the performance of EDICT and BDIA-DDIM for (round-trip) image editing.

BDIA-DDIM inversion: In brief, to allow for round-trip data manipulation such as image editing, the first step is to start from a clean image $\mathbf{z}_0 = \mathbf{x}$ and perform BDIA-DDIM inversion iteratively (except the computation of $\mathbf{z}_1$) to

Algorithm 2 Round-trip image-editing procedure of BDIA-DDIM

```

1: Input:  $\mathbf{z}_0 = \mathbf{x}$ ,  $N$ ,  $\phi$ ,  $\tilde{\phi}$ ,  $\gamma \in (0, 1]$ 
2:  $\mathbf{z}_1 = \left( \frac{\mathbf{z}_0}{a_1} - \frac{b_1}{a_1} \hat{\epsilon}_{\theta}(\mathbf{z}_0, \phi, 0) \right)$ 
3: for  $i = 1, \dots, N - 1$  do
4:    $\mathbf{z}_{i+1} = \mathbf{z}_{i-1}/\gamma - \left[ a_i \mathbf{z}_i + b_i \hat{\epsilon}_{\theta}(\mathbf{z}_i, \phi, i) \right] / \gamma + \left[ \frac{\mathbf{z}_i}{a_{i+1}} - \frac{b_{i+1}}{a_{i+1}} \hat{\epsilon}_{\theta}(\mathbf{z}_i, \phi, i) \right]$ 
5: end for
6: Set  $(\tilde{\mathbf{z}}_N, \tilde{\mathbf{z}}_{N-1}) = (\mathbf{z}_N, \mathbf{z}_{N-1})$ 
7: for  $i = N - 1, \dots, 1$  do
8:    $\tilde{\mathbf{z}}_{i-1} = \gamma \tilde{\mathbf{z}}_{i+1} - \gamma \left[ \frac{\tilde{\mathbf{z}}_i}{a_{i+1}} - \frac{b_{i+1}}{a_{i+1}} \hat{\epsilon}_{\theta}(\tilde{\mathbf{z}}_i, \tilde{\phi}, i) \right] + \left[ a_i \tilde{\mathbf{z}}_i + b_i \hat{\epsilon}_{\theta}(\tilde{\mathbf{z}}_i, \tilde{\phi}, i) \right]$ 
9: end for
10: Output:  $\tilde{\mathbf{z}}_0$ 

```

compute a final pair of noise representations $(\mathbf{z}_N, \mathbf{z}_{N-1})$, conditioned on “reasonable” side information ϕ , such as text input correctly describing the image. Upon obtaining $(\mathbf{z}_N, \mathbf{z}_{N-1})$, we apply BDIA-DDIM based on the modified side information $\tilde{\phi}$ to acquire the final edited data $\tilde{\mathbf{z}}_0 = \tilde{\mathbf{x}}$.

We first explain the process for obtaining $(\mathbf{z}_N, \mathbf{z}_{N-1})$ given $\mathbf{z}_0$ and ϕ . To start, we apply DDIM inversion to compute $\mathbf{z}_1$ with $\mathbf{z}_0$ and ϕ . After that, we compute $\mathbf{z}_{i+1}$ iteratively given $(\mathbf{z}_i, \mathbf{z}_{i-1})$ by following BDIA-DDIM inversion, where $i = 1, \dots, N - 1$. It follows from (24) that the diffusion state $\mathbf{z}_{i+1}$ can be computed in terms of $(\mathbf{z}_i, \mathbf{z}_{i-1})$ as

$$\mathbf{z}_{i+1} = \mathbf{z}_{i-1}/\gamma - \left[a_i \mathbf{z}_i + b_i \hat{\epsilon}_{\theta}(\mathbf{z}_i, \phi, i) \right] / \gamma + \left(\frac{\mathbf{z}_i}{a_{i+1}} - \frac{b_{i+1}}{a_{i+1}} \hat{\epsilon}_{\theta}(\mathbf{z}_i, \phi, i) \right), \quad (28)$$

where ϕ is the additional conditioning input to the neural network θ . Similarly to the computation (28), EDICT also does not involve any approximation and results in exact diffusion inversion. However, in contrast to EDICT, (28) does not require a doubling of the NFE. Intuitively speaking, the efficiency of BDIA is because our new technique exploits time-reversibility while EDICT enforces reversibility by introducing an auxiliary diffusion state per timestep. For the special case of $\gamma = 1$, both (25) and (28) are symmetric in time (provided the conditioning input ϕ is also symmetric). That is, switching the timestep t_{i+1} and t_{i-1} in (25) or (28) with $\gamma = 1$ inverts the diffusion direction. We summarize the above property of time-symmetry in a lemma:

Proposition 2 (time-symmetry). *Switching the timestep t_{i-1} and t_{i+1} in (25) produces the reverse update (28) for the case $\gamma = 1$, and vice versa.*

Once we obtain $(\mathbf{z}_N, \mathbf{z}_{N-1}) = (\tilde{\mathbf{z}}_N, \tilde{\mathbf{z}}_{N-1})$, the remaining step is to apply BDIA-DDIM iteratively until reaching $\tilde{\mathbf{z}}_0$ by using the modified side information $\tilde{\phi}$. To do so, one can simply apply (24) by replacing $\hat{\epsilon}_{\theta}(\tilde{\mathbf{z}}_i, i)$ with $\hat{\epsilon}_{\theta}(\tilde{\mathbf{z}}_i, \tilde{\phi}, i)$. Note that when $\tilde{\phi} = \phi$, the output $\tilde{\mathbf{z}}_0$ becomes a reconstruction of the original clean image $\mathbf{z}_0 = \mathbf{x}$. See Alg. 2 for the (round-trip) image-editing procedure of BDIA-DDIM.

Remark 1. We have also designed coupled BDIA (CBDIA) as an extension of EDICT. CBDIA is also reversible and requires two NFEs per timestep in general. CBDIA includes BDIA with $\gamma = 1$ as a special case. See Appendix D for details.

Convergence analysis of BDIA-DDIM: Before we formally present the convergence results for BDIA-DDIM, we first introduce some notations and present two assumptions for $\hat{\epsilon}_\theta$. We use $\hat{\epsilon}_\theta[m]$ to denote the m th component of $\hat{\epsilon}_\theta$. To facilitate the analysis later on, we denote the discrete stepsize at time t_i to be

$$h = \lambda_i - \lambda_{i-1} > 0 \quad \text{where } \lambda_j = \lambda_{t_j} = \frac{\sigma_{t_j}}{\alpha_{t_j}} \quad j = i, i-1 \quad (29)$$

for all $i = 1, \dots, N$, over the time interval $t \in [T, \epsilon]$, where in practice, ϵ is set to be $\epsilon > 0$. For consistency, we let $\lambda_N = \lambda_T$ and $\lambda_0 = \lambda_\epsilon$. The two assumptions regarding $\hat{\epsilon}_\theta$ are summarized as below:

Assumption 1 Assume $\hat{\epsilon}_\theta(\mathbf{z}, t)$ satisfies the Lipschitz condition

$$\|\hat{\epsilon}_\theta(\mathbf{z}, t) - \hat{\epsilon}_\theta(\mathbf{y}, t)\|_1 \leq K_1 \|\mathbf{z} - \mathbf{y}\|_1 \quad T \geq t \geq \epsilon \quad \forall \mathbf{z}, \mathbf{y} \in \mathbb{R}^d, \quad (30)$$

where $K_1 > 0$, and d denotes the dimensionality of $\hat{\epsilon}_\theta$.

Assumption 2 Let $\hat{\epsilon}'_\theta(\mathbf{z}, t)$ and λ'_t denote the differentials of $\hat{\epsilon}_\theta(\mathbf{z}, t)$ and λ_t with respect to t for $T \geq t \geq \epsilon$. Assume the magnitudes of $\{(1/\lambda'_t)\hat{\epsilon}'_\theta(\mathbf{z}, t)[m]\}_{m=1}^d$ are upper-bounded, represented as

$$\left| \frac{1}{\lambda'_t} \hat{\epsilon}'_\theta(\mathbf{z}, t)[m] \right| \leq K_2 \quad T \geq t \geq \epsilon \quad \forall \mathbf{z} \in \mathbb{R}^d \quad m = 1, \dots, d. \quad (31)$$

Theorem 1. Under the two Assumptions 1-2, consider using BDIA-DDIM to solve the ODE as specified by (2) and (4) with a uniform discrete step h as defined in (29). Denote the ground truth of the diffusion states of the ODE as $\mathbf{z}_i^* = \mathbf{z}_{t_i}^*$ over the time interval $[T, \epsilon]$. Let the initial errors with regard to the initial states $\{\mathbf{z}_i | i = N, N-1\}$ satisfy

$$\eta(h) = \max_{N \geq i \geq N-1} \|\mathbf{z}_i^* - \mathbf{z}_i\|_1 \rightarrow 0 \quad \text{as } h \rightarrow 0. \quad (32)$$

Then the error $\|\mathbf{z}_i^* - \mathbf{z}_i\|_1$, $N-2 \geq i \geq 0$, is upper-bounded by

$$\|\mathbf{z}_i^* - \mathbf{z}_i\|_1 \leq e^{\beta(\lambda_N - \lambda_0)} \eta(h) + \frac{\alpha_0 + \gamma\alpha_2}{2\beta} K_2 d (1 - e^{\beta(\lambda_N - \lambda_0)}) h, \quad (33)$$

where $\beta = \frac{2}{\lambda_0} + (\alpha_0 + \gamma\alpha_2)K_1$ and d denotes the dimensionality of $\hat{\epsilon}_\theta$. It is straightforward from (32) and (33) that the update expression of BDIA-DDIM converges in the time limit, i.e.,

$$\|\mathbf{z}_i^* - \mathbf{z}_i\|_1 \rightarrow 0 \quad \text{as } h \rightarrow 0 \quad \forall i = 0, \dots, N-2. \quad (34)$$

Proof. See proof in Appendix B. Our convergence argument is inspired by the proof for Theorem 6.6 in [2] for linear multi-step methods.

When setting $\gamma = 0$ in Theorem 1, we obtain the convergence results for DDIM. Considering the initial condition (32), one can easily show by following Appendix B that it holds when applying DDIM in computing $\mathbf{z}_{N-1}$ given $\mathbf{z}_N$.

5 Experiments

We conducted two types of experiments: (1) image sampling; (2) image-editing. It was found that our new technique BDIA produces promising results for both tasks.

5.1 Evaluation of image sampling

Text-to-image generation: In this task, we tested five sampling methods by using StableDiffusion V2⁶ which are DDIM, DPM-Solver++, PLMS, EDICT, and BDIA-DDIM. COCO2014 validation set was utilized for this task. For each method, 20K images of size 512×512 were generated by feeding 20K text prompts to the diffusion model. All the five methods share the same seed of the random noise generator and the same text prompts. The FID score for each method was computed by resizing the generated images to size of 256×256 due to the fact that the original images in COCO2014 in general have lower resolution than the size of 512×512 .

Table 1: FID ($\downarrow$) and CLIP ($\uparrow$) evaluation of five methods for text-to-image generation over StableDiffusion V2. See also Appendix H for how p and γ affect the sampling performance of EDICT and BDIA-DDIM, respectively. For each number of timesteps, EDICT requires twice the NFEs needed in each of other four methods. The results of EDICT at 80 NFEs are better than the results of DDIM at 40 NFEs, which are consistent with those of [40]. The values denoted with * are taken from [42].

		EDICT	BDIA-DDIM	DDIM	DPM-Solver++	PLMS
10 NFEs	FID	-	12.62	14.78*	15.82*	24.42*
	CLIP	-	25.05	24.86*	24.83*	23.85*
20 NFEs	FID	30.91	13.44	14.65*	13.85*	14.30*
	CLIP	24.98	25.12	25.00*	25.02*	24.80*
40 NFEs	FID	15.51	13.87	14.69*	14.29*	15.05*
	CLIP	25.44	25.05	25.01*	25.05*	24.95*
80 NFEs	FID	14.34	13.53	15.16	14.83	14.97
	CLIP	25.11	24.85	24.99	25.04	25.00

It is clear from Table 1 that BDIA-DDIM has a remarkable FID performance gain over the other four methods. This indicates that the introduced backward DDIM update per timestep indeed helps to improve the accuracy of the corresponding integration approximation. EDICT, on the other hand, produces better CLIP scores for 40 and 80 NFEs. The visual improvement of BDIA-DDIM over DDIM is also demonstrated in Figs. 1, 7, 8.

Conventional image generation: In this experiment, we consider the task of noise-to-image generation without conditioning text. The parameter γ in BDIA-DDIM was set to $\gamma = 1.0$. The tested pre-trained models can be found in Appendix G. Given a pre-trained model, 50K artificial images were generated for a particular NFE, and the corresponding FID score was computed.

Table 2 summarize the computed FID scores. It is clear that by incorporating BDIA into DDIM, the FID scores are improved considerably. This can be explained by the fact that BDIA introduces the additional backward integration approximation per time-step in the sampling process. This makes the

⁶ <https://github.com/Stability-AI/stablediffusion>

Table 2: FID comparison for conventional sampling. See Appendix I for how p affects the sampling performance of EDICT. Recall that for each number of timesteps, EDICT requires twice the number of NFEs needed by DDIM and BDIA-DDIM.

NFEs		EDICT	DDIM	BDIA-DDIM		EDICT	DDIM	BDIA-DDIM
10	CIFAR10	-	14.38	10.03	CelebA	-	13.41	10.86
20		203.86	7.51	6.29		190.62	9.45	8.86
40		116.72	4.95	4.63		81.44	6.93	6.50
80		51.17	3.82	3.70		43.93	4.87	4.65

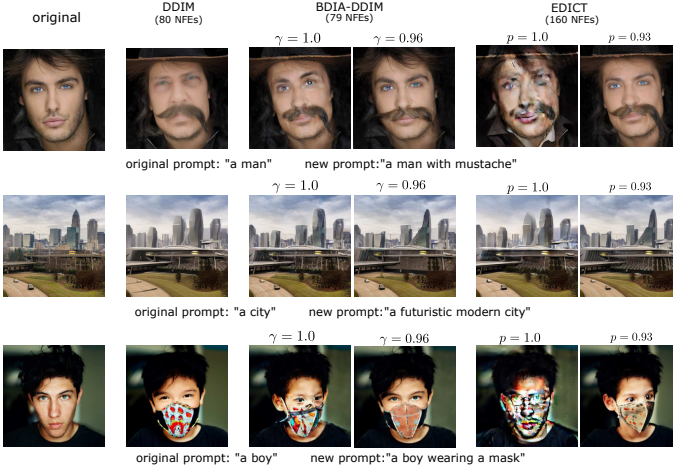


Fig. 5: Round-trip image editing via DDIM, BDIA-DDIM and EDICT. $p = 0.93$ is the recommended setup in [40] for EDICT. Similarly to Fig. 3, the number of timesteps was set to 40 in all the three methods. See Appendix K for additional text-based and ControlNet-based image-editing results.

resulting final integration approximation more accurate. We have also provided an explanation in appendix I why EDICT does not perform well in Table 2.

5.2 Evaluation of image-editing

In this second experiment, we evaluated BDIA-DDIM for round-trip image-editing by utilizing the open-source repository of EDICT,⁷ and for single-trip ControlNet-based image-editing by using the repository of ControlNet.⁸ ControlNet [43] allows the spatial layout of a synthesized image to be controlled with conditioning in the form of edges, “scribbles”, and other modalities. In the experiment, we utilize the scribble format for image editing. Fig. 5 and 6 visualize the obtained results. We point out that in Fig. 5, BDIA-DDIM produces very similar results to EDICT while reducing by half the NFE compared to EDICT. Fig. 6 demonstrates that BDIA-DDIM produces better image quality than DDIM for ControlNet-based image editing.

⁷ <https://github.com/salesforce/EDICT>

⁸ <https://github.com/llyasviel/ControlNet>



Fig. 6: Single-trip ControlNet-based image editing by using DDIM and BDIA-DDIM at 10 timesteps. The hyper-parameter γ in BDIA-DDIM was set to 0.5. See Appendix K for additional ControlNet-based image-editing results.

5.3 Editability and limitations of BDIA-DDIM

Figs. 3 and 5 demonstrate different effects that are obtained by changing the γ parameter. As image editing is inherently subjective and the algorithm cannot guess what effect is desired, the availability of this parameter can be viewed as an advantage of BDIA-DDIM. In practice, the γ parameter can be adjusted by the artist to produce different image-editing effects. Since GPUs allow the processing of a batch of images with different γ values simultaneously, the artist can thus choose the generated image with the desired effect from the batch.

In comparison to DDIM, BDIA-DDIM requires additional memory to store $\mathbf{z}_{i+1}$ in computing $\mathbf{z}_{i-1}$. However, it produces a better image quality. The additional computational overhead in BDIA-DDIM can be ignored since only simple linear algebra is involved. When performing round-trip image editing, the update expression for BDIA-DDIM inversion requires two initial diffusion states $(\mathbf{z}_0, \mathbf{z}_1)$, where $\mathbf{z}_1$ must be computed by following other methods, such as DDIM.

6 Conclusions

In this paper, we have proposed a new technique BDIA, to assist DDIM in achieving exact diffusion inversion. The key step of BDIA-DDIM is to perform DDIM update procedure twice at each time step t_i : one over the previous time-slot $[t_i, t_{i+1}]$ and the other over the next time-slot $[t_i, t_{i-1}]$ in computing $\mathbf{z}_{i-1}$. By doing so, the expression for $\mathbf{z}_{i-1}$ becomes a linear combination of $(\mathbf{z}_i, \hat{\epsilon}_\theta(\mathbf{z}_i, i), \mathbf{z}_{i+1})$. As a result, $\mathbf{z}_{i+1}$ can be computed exactly as a linear function of $(\mathbf{z}_i, \hat{\epsilon}_\theta(\mathbf{z}_i, i), \mathbf{z}_{i-1})$, enabling exact diffusion inversion. Note that although the DDIM update is evaluated twice at each step, this is inexpensive since the costly neural functional evaluation is performed only once. Convergence results for BDIA-DDIM are provided by following an analysis similar to that for linear multi-step methods. Extensive experiments show that BDIA-DDIM not only enables effective image editing but also improves over the image sampling quality of DDIM and EDICT. Please see Appendix K in the supplementary material for additional text-based and ControlNet-based image-editing results.

References

1. Arjovsky, M., Chintala, S., Bottou, L.: Wasserstein GAN. arXiv:1701.07875 [stat.ML] (2017) **1**
2. Atkinson, K.E.: An Introduction to Numerical Analysis. John Wiley Sons, 2nd edn. (1988) **3, 11, 21**
3. Bao, F., Li, C., Sun, J., Zhu, J., Zhang, B.: Estimating the Optimal Covariance with Imperfect Mean in Diffusion Probabilistic Models. In: ICML (2022) **1**
4. Bao, F., Li, C., Zhu, J., Zhang, B.: Analytic-DPM: an Analytic Estimate of the Optimal Reverse Variance in Diffusion Probabilistic Models. In: ICLR (2022) **1**
5. Bishop, C.M.: Pattern Recognition and Machine Learning. Springer (2006) **1**
6. Chen, N., Zhang, Y., Zen, H., Weiss, R.J., Norouzi, M., Chan, W.: WaveGrad: Estimating Gradients for Waveform Generation. arXiv:2009.00713 (September 2020) **1**
7. Dhariwal, P., Nichol, A.: Diffusion models beat gans on image synthesis. arXiv:2105.05233 [cs.LG] (2021) **1**
8. Dinh, L., Krueger, D., Bengio, Y.: Nice: Non-linear independent components estimation. arXiv preprint arXiv:1410.8516 (2014) **2**
9. Dinh, L., Sohl-Dickstein, J., Bengio, S.: Density estimation using real nvp. arXiv preprint arXiv:1605.08803 (2016) **2**
10. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y.: Generative Adversarial Nets. In: Proceedings of the International Conference on Neural Information Processing Systems. pp. 2672–2680 (2014) **1**
11. Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., Courville, A.C.: Improved training of wasserstein gans. In: Advances in neural information processing systems. pp. 5767–5777 (2017) **1**
12. Ho, J., Jain, A., Abbeel, P.: Denoising diffusion probabilistic models. In: NeurIPS (2020) **1, 4**
13. Ho, J., Salimans, T.: Classifier-free diffusion guidance. arXiv preprint arXiv:2207.12598 (2022) **1**
14. Huberman-Spiegelglas, I., Kulikov, V., Michaeli, T.: An Edit Friendly DDPM Noise Space: Inversion and Manipulations. arXiv:2304.06140v2 [cs.CV] (2023) **2**
15. Hyvarinen, A.: Estimation of non-normalized statistical models by score matching. Journal of Machine Learning Research **24**, 695–709 (2005) **4**
16. Karras, T., Aittala, M., Alia, T., Laine, S.: Elucidating the Design Space of Diffusion-Based Generative Models. In: 36th Conference on Neural Information Processing Systems (NeurIPS) (2022) **1, 4, 25, 26**
17. Kim, D., Kim, Y., Kwon, S.J., Kang, W., Moon, I.C.: Refining Generative Process with Discriminator Guidance in Score-based Diffusion Models. arXiv preprint arXiv:2211.17091 [cs.CV] (2022) **1**
18. Kingma, D.P., Dhariwal, P.: Glow: Generative flow with invertible 1x1 convolutions. In: Advances in neural information processing systems (2018) **2, 6**
19. Kingma, D.P., Salimans, T., Poole, B., Ho, J.: Variational diffusion models. arXiv: preprint arXiv:2107.00630 (2021) **1**
20. Lam, M.W.Y., Wang, J., Su, D., Yu, D.: BDDM: Bilateral Denoising Diffusion Models for Fast and High-Quality Speech Synthesis. In: ICLR (2022) **1**
21. Liu, L., Ren, Y., Lin, Z., Zhao, Z.: Pseudo Numerical Methods for Diffusion Models on Manifolds. In: ICLR (2022) **1**

22. Lu, C., Zhou, Y., Bao, F., Chen, J., Li, C., Zhu, J.: DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Sampling in Around 10 Steps. In: NeurIPS (2022) [3](#), [4](#), [5](#), [21](#)
23. Lu, C., Zhou, Y., Bao, F., Chen, J., Li, C., Zhu, J.: DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models. arXiv preprint arXiv:2211.01095 [cs.LG] (November 2022) [1](#)
24. Mokady, R., Hertz, A., Aberman, K., Pritch, Y., Cohen-Or, D.: Null-text Inversion for Editing Real Images using Guided Diffusion Models. In: CVPR (2023) [1](#), [2](#)
25. Nichol, A., Dhariwal, P.: Improved denoising diffusion probabilistic models. arXiv preprint arXiv:2102.09672 (2021) [1](#)
26. Nichol, A., Dhariwal, P., Ramesh, A., Shyam, P., Mishkin, P., McGrew, B., Sutskever, I., Chen, M.: GLIDE: Towards Photorealistic image generation and editing with text-guided diffusion models. In: ICML (2022) [1](#)
27. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., Ommer, B.: High-resolution image synthesis with latent diffusion models. In: CVPR (2022) [1](#)
28. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., Ommer, B.: On High-resolution image synthesis with latent diffusion models. In: CVPR. p. 10684–10695 (2022) [2](#)
29. Ronneberger, O., Fischer, P., Brox, T.: U-Net: Convolutional Networks for Biomedical Image Segmentation. arXiv:1505.04597 [cs.CV] (2015) [4](#)
30. Saharia, C., Chan, W., Saxena, S., Li, L., Whang, J., Denton, E., Ghasemipour, S.K.S., Ayan, B.K., Mahdavi, S.S., Lopes, R.G., Salimans, T., Ho, J., Fleet, D.J., Norouzi, M.: Photorealistic text-to-image diffusion models with deep language understanding. arXiv preprint arXiv:2205.11487 (2022) [2](#)
31. Sauer, A., Schwarz, K., Geiger, A.: StyleGAN-XL: Scaling StyleGAN to large diverse datasets. In: SIGGRAPH (2022) [1](#)
32. Shi, Y., Xue, C., Pan, J., Zhang, W.: DragDiffusion: Harnessing Diffusion Models for Interactive Point-based Image Editing. arXiv:2306.14435v2 (2023) [2](#)
33. Skeel, R.D.: Variable Step Size Destabilizes the Störmer/Leapfrog/Verlet Method. BIT Numerical Mathematics **33**, 172–175 (1993) [3](#)
34. Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., Ganguli, S.: Deep unsupervised learning using nonequilibrium thermodynamics. ICML (2015) [1](#)
35. Song, J., Meng, C., Ermon, S.: Denoising Diffusion Implicit Models. In: ICLR (2021) [1](#), [4](#)
36. Song, Y., Durkan, C., Murray, I., Ermon, S.: Maximum likelihood training of score-based diffusion models. In: Advances in neural information processing systems (NeurIPS) (2021) [1](#)
37. Song, Y., Ermon, S.: Generative modeling by estimating gradients of the data distribution. In: Advances in neural information processing systems (NeurIPS). p. 11895–11907 (2019) [1](#)
38. Song, Y., S.-Dickstein, J., Kingma, D.P., Kumar, A., Ermon, S., Poole, B.: Score-Based Generative Modeling Through Stochastic Differential Equations. In: ICLR (2021) [1](#), [4](#)
39. Verlet, L.: Computer Experiments on Classical Fluids. I. Thermodynamical Properties of Lennard-Jones Molecules. Physical Review **159**(1), 98–103 (1967) [3](#)
40. Wallace, B., Gokul, A., Naik, N.: EDICT: Exact Diffusion Inversion via Coupled Transformations. In: CVPR (2023) [1](#), [2](#), [6](#), [12](#), [13](#), [31](#), [37](#), [38](#)
41. Xu, Y., Deng, M., Cheng, X., Tian, Y., Liu, Z., Jaakkola, T.: Restart Sampling for Improving Generative Processes. In: NeurIPS (2023) [3](#)
42. Zhang, G., Niwa, K., Kleijn, W.B.: On Accelerating Diffusion-Based Sampling Processes by Improved Integration Approximation. ICLR (2024) [1](#), [4](#), [12](#)

43. Zhang, L., Rao, A., Agrawala, M.: Adding conditional control to text-to-image diffusion models. In: IEEE International Conference on Computer Vision (ICCV) (2023) [13](#)
44. Zhang, Q., Chenu, Y.: Fast Sampling of Diffusion Models with Exponential Integrator. arXiv:2204.13902 [cs.LG] (2022) [1](#), [5](#)

A Proof for Proposition 1

We prove Proposition 1 by induction. To do so, we first compute the update expression for $\mathbf{z}_{N-2}$. It is known from (22) that

$$\mathbf{z}_{N-1} = \mathbf{z}_N + \Delta(t_N \rightarrow t_{N-1} | \mathbf{z}_N).$$

With the expression for $\mathbf{z}_{N-1}$, $\mathbf{z}_{N-2}$ can then be computed by using (24):

$$\begin{aligned} \mathbf{z}_{N-2} &= \mathbf{z}_N - (1-\gamma)(\mathbf{z}_N - \mathbf{z}_{N-1}) - \gamma\Delta(t_{N-1} \rightarrow t_N | \mathbf{z}_{N-1}) + \Delta(t_{N-1} \rightarrow t_{N-2} | \mathbf{z}_{N-1}) \\ &= \mathbf{z}_N + (1-\gamma)\Delta(t_N \rightarrow t_{N-1} | \mathbf{z}_N) - \gamma\Delta(t_{N-1} \rightarrow t_N | \mathbf{z}_{N-1}) \\ &\quad + \Delta(t_{N-1} \rightarrow t_{N-2} | \mathbf{z}_{N-1}) \end{aligned}$$

which satisfies (27). Furthermore, the difference $\mathbf{z}_{N-1} - \mathbf{z}_{N-2}$ can be represented as

$$\begin{aligned} \mathbf{z}_{N-1} - \mathbf{z}_{N-2} &= \gamma\Delta(t_N \rightarrow t_{N-1} | \mathbf{z}_N) + \gamma\Delta(t_{N-1} \rightarrow t_N | \mathbf{z}_{N-1}) \\ &\quad - \Delta(t_{N-1} \rightarrow t_{N-2} | \mathbf{z}_{N-1}) \end{aligned} \tag{35}$$

Next, we show that the expression for $\mathbf{z}_{N-3}$ also takes the form of (27). Again by using (24), $\mathbf{z}_{N-3}$ can be computed to be

$$\begin{aligned} \mathbf{z}_{N-3} &= \mathbf{z}_{N-1} - (1-\gamma)(\mathbf{z}_{N-1} - \mathbf{z}_{N-2}) - \gamma\Delta(t_{N-2} \rightarrow t_{N-1} | \mathbf{z}_{N-2}) \\ &\quad + \Delta(t_{N-2} \rightarrow t_{N-3} | \mathbf{z}_{N-2}) \\ &\stackrel{(a)}{=} \mathbf{z}_{N-1} - (1-\gamma) \left[\gamma\Delta(t_N \rightarrow t_{N-1} | \mathbf{z}_N) + \gamma\Delta(t_{N-1} \rightarrow t_N | \mathbf{z}_{N-1}) \right. \\ &\quad \left. - \Delta(t_{N-1} \rightarrow t_{N-2} | \mathbf{z}_{N-1}) \right] \\ &\quad - \gamma\Delta(t_{N-2} \rightarrow t_{N-1} | \mathbf{z}_{N-2}) + \Delta(t_{N-2} \rightarrow t_{N-3} | \mathbf{z}_{N-2}) \\ &= \mathbf{z}_N + [1 - (1-\gamma)\gamma]\Delta(t_N \rightarrow t_{N-1} | \mathbf{z}_N) - (1-\gamma)\gamma\Delta(t_{N-1} \rightarrow t_N | \mathbf{z}_{N-1}) \\ &\quad + (1-\gamma)\Delta(t_{N-1} \rightarrow t_{N-2} | \mathbf{z}_{N-1}) - \gamma\Delta(t_{N-2} \rightarrow t_{N-1} | \mathbf{z}_{N-2}) \\ &\quad + \Delta(t_{N-2} \rightarrow t_{N-3} | \mathbf{z}_{N-2}) \\ &= \mathbf{z}_N + \frac{1 - (-\gamma)^3}{1 + \gamma} \Delta(t_N \rightarrow t_{N-1} | \mathbf{z}_N) - \frac{\gamma(1 - \gamma^2)}{1 + \gamma} \Delta(t_{N-1} \rightarrow t_N | \mathbf{z}_{N-1}) \\ &\quad + (1-\gamma)\Delta(t_{N-1} \rightarrow t_{N-2} | \mathbf{z}_{N-1}) - \gamma\Delta(t_{N-2} \rightarrow t_{N-1} | \mathbf{z}_{N-2}) \\ &\quad + \Delta(t_{N-2} \rightarrow t_{N-3} | \mathbf{z}_{N-2}) \\ &= \mathbf{z}_N + \sum_{j=N-1}^N \left[\frac{1 - (-\gamma)^{j-i}}{1 + \gamma} \Delta(t_j \rightarrow t_{j-1} | \mathbf{z}_j) \right. \\ &\quad \left. - \frac{\gamma + (-\gamma)^{j-i}}{1 + \gamma} \Delta(t_{j-1} \rightarrow t_j | \mathbf{z}_{j-1}) \right] \\ &\quad + \Delta(t_{N-2} \rightarrow t_{N-3} | \mathbf{z}_{N-2}). \end{aligned}$$

where step (a) makes use of (35). It is clear that the above expression for $\mathbf{z}_{N-3}$ is identical to (27) with $i = N - 3$.

The final step is to first assume that $\mathbf{z}_k$ and $\mathbf{z}_{k+1}$ can be represented by (27) with $i = k$ and $i = k + 1$, respectively. That is,

$$\begin{aligned} \mathbf{z}_k = \mathbf{z}_N + \sum_{j=k+2}^N & \left[\frac{1 - (-\gamma)^{j-k}}{1 + \gamma} \Delta(t_j \rightarrow t_{j-1} | \mathbf{z}_j) \right. \\ & \left. - \frac{\gamma + (-\gamma)^{j-k}}{1 + \gamma} \Delta(t_{j-1} \rightarrow t_j | \mathbf{z}_{j-1}) \right] \\ & + \Delta(t_{k+1} \rightarrow t_k | \mathbf{z}_{k+1}) \end{aligned} \quad (36)$$

$$\begin{aligned} \mathbf{z}_{k+1} = \mathbf{z}_N + \sum_{j=k+3}^N & \left[\frac{1 - (-\gamma)^{j-k-1}}{1 + \gamma} \Delta(t_j \rightarrow t_{j-1} | \mathbf{z}_j) \right. \\ & \left. - \frac{\gamma + (-\gamma)^{j-k-1}}{1 + \gamma} \Delta(t_{j-1} \rightarrow t_j | \mathbf{z}_{j-1}) \right] \\ & + \Delta(t_{k+2} \rightarrow t_{k+1} | \mathbf{z}_{k+2}). \end{aligned} \quad (37)$$

We will then show in the following that $\mathbf{z}_{k-1}$ is again identical to (27) with $i = k - 1$. From (24), $\mathbf{z}_{k-1}$ can be computed in terms of $(\mathbf{z}_k, \mathbf{z}_{k+1})$ as

$$\begin{aligned} \mathbf{z}_{k-1} &= \mathbf{z}_{k+1} - (1 - \gamma)(\mathbf{z}_{k+1} - \mathbf{z}_k) - \gamma \Delta(t_k \rightarrow t_{k+1} | \mathbf{z}_k) + \Delta(t_k \rightarrow t_{k-1} | \mathbf{z}_k) \\ &= \gamma \mathbf{z}_{k+1} + (1 - \gamma) \mathbf{z}_k - \gamma \Delta(t_k \rightarrow t_{k+1} | \mathbf{z}_k) + \Delta(t_k \rightarrow t_{k-1} | \mathbf{z}_k) \end{aligned} \quad (38)$$

Plugging (37) and (36) into (38) produces

$$\begin{aligned} \mathbf{z}_{k-1} &= \gamma \mathbf{z}_N + \gamma \sum_{j=k+3}^N \left[\frac{1 - (-\gamma)^{j-k-1}}{1 + \gamma} \Delta(t_j \rightarrow t_{j-1} | \mathbf{z}_j) \right. \\ & \quad \left. - \frac{\gamma + (-\gamma)^{j-k-1}}{1 + \gamma} \Delta(t_{j-1} \rightarrow t_j | \mathbf{z}_{j-1}) \right] \\ & \quad + \gamma \Delta(t_{k+2} \rightarrow t_{k+1} | \mathbf{z}_{k+2}) \\ & \quad + (1 - \gamma) \mathbf{z}_N + (1 - \gamma) \sum_{j=k+2}^N \left[\frac{1 - (-\gamma)^{j-k}}{1 + \gamma} \Delta(t_j \rightarrow t_{j-1} | \mathbf{z}_j) \right. \\ & \quad \left. - \frac{\gamma + (-\gamma)^{j-k}}{1 + \gamma} \Delta(t_{j-1} \rightarrow t_j | \mathbf{z}_{j-1}) \right] \\ & \quad + (1 - \gamma) \Delta(t_{k+1} \rightarrow t_k | \mathbf{z}_{k+1}) \\ & \quad - \gamma \Delta(t_k \rightarrow t_{k+1} | \mathbf{z}_k) + \Delta(t_k \rightarrow t_{k-1} | \mathbf{z}_k) \end{aligned}$$

$$\begin{aligned}
&= \mathbf{z}_N + \sum_{j=k+3}^N \left[\frac{(\gamma + (-\gamma)^{j-k}) + (1-\gamma)(1 - (-\gamma)^{j-k})}{1+\gamma} \Delta(t_j \rightarrow t_{j-1} | \mathbf{z}_j) \right. \\
&\quad \left. - \frac{\gamma^2 - (-\gamma)^{j-k} + (1-\gamma)(\gamma + (-\gamma)^{j-k})}{1+\gamma} \Delta(t_{j-1} \rightarrow t_j | \mathbf{z}_{j-1}) \right] \\
&+ \left[\frac{\gamma + \gamma^2 + (1-\gamma)(1 - (-\gamma)^2)}{1+\gamma} \Delta(t_{k+2} \rightarrow t_{k+1} | \mathbf{z}_{k+2}) \right. \\
&\quad \left. - \frac{(1-\gamma)(\gamma + (-\gamma)^2)}{1+\gamma} \Delta(t_{k+1} \rightarrow t_{k+2} | \mathbf{z}_{k+1}) \right] \\
&+ (1-\gamma) \Delta(t_{k+1} \rightarrow t_k | \mathbf{z}_{k+1}) - \gamma \Delta(t_k \rightarrow t_{k+1} | \mathbf{z}_k) \\
&+ \Delta(t_k \rightarrow t_{k-1} | \mathbf{z}_k) \\
&= \mathbf{z}_N + \sum_{j=k+3}^N \left[\frac{1 - (-\gamma)^{j-k+1}}{1+\gamma} \Delta(t_j \rightarrow t_{j-1} | \mathbf{z}_j) \right. \\
&\quad \left. - \frac{\gamma + (-\gamma)^{j-k+1}}{1+\gamma} \Delta(t_{j-1} \rightarrow t_j | \mathbf{z}_{j-1}) \right] \\
&+ \left[\frac{1 - (-\gamma)^3}{1+\gamma} \Delta(t_{k+2} \rightarrow t_{k+1} | \mathbf{z}_{k+2}) - \frac{\gamma + (-\gamma)^3}{1+\gamma} \Delta(t_{k+1} \rightarrow t_{k+2} | \mathbf{z}_{k+1}) \right] \\
&+ (1-\gamma) \Delta(t_{k+1} \rightarrow t_k | \mathbf{z}_{k+1}) - \gamma \Delta(t_k \rightarrow t_{k+1} | \mathbf{z}_k) \\
&+ \Delta(t_k \rightarrow t_{k-1} | \mathbf{z}_k) \\
&= \mathbf{z}_N + \sum_{j=k+1}^N \left[\frac{1 - (-\gamma)^{j-k+1}}{1+\gamma} \Delta(t_j \rightarrow t_{j-1} | \mathbf{z}_j) \right. \\
&\quad \left. - \frac{\gamma + (-\gamma)^{j-k+1}}{1+\gamma} \Delta(t_{j-1} \rightarrow t_j | \mathbf{z}_{j-1}) \right] \\
&+ \Delta(t_k \rightarrow t_{k-1} | \mathbf{z}_k) \tag{39}
\end{aligned}$$

We can confirm from (39) that $\mathbf{z}_{k-1}$ is indeed identical to (27) with $i = k - 1$. The proof is complete.

A.1 The special case of $\gamma = 1$

Corollary 1. *For the special case of $\gamma = 1$, the update expression (27) for BDIA-DDIM in Proposition 1 can be simplified to be*

$$\begin{aligned}
\mathbf{z}_i &= \mathbf{z}_N + \Delta(t_N \rightarrow t_{N-1} | \mathbf{z}_N) \bmod (N-i, 2) \\
&+ \sum_{j=i+1}^{N-1} (-\Delta(t_j \rightarrow t_{j+1} | \mathbf{z}_j) + \Delta(t_j \rightarrow t_{j-1} | \mathbf{z}_j)) \bmod (j-i, 2). \tag{40}
\end{aligned}$$

A.2 Higher-order BDIA

One can easily design high-order BDIA solvers as an extension of (24). For instance, $\mathbf{z}_{i-1}$ can be computed in terms of $(\mathbf{z}_i, \mathbf{z}_{i+1}, \mathbf{z}_{i+2})$ as

$$\begin{aligned}
 \mathbf{z}_{i-1} = & \underbrace{\mathbf{z}_{i+2} - (1-\gamma_2)(\mathbf{z}_{i+2} - \mathbf{z}_{i+1}) - \gamma_2 \Delta(t_{i+1} \rightarrow t_{i+2} | \mathbf{z}_i, \mathbf{z}_{i+1})}_{\approx \int_{t_{i+2}}^{t_{i+1}} \mathbf{d}(\mathbf{z}_\tau, \tau) d\tau} \\
 & \underbrace{-(1-\gamma_1)(\mathbf{z}_{i+1} - \mathbf{z}_i) - \gamma_1 \Delta(t_i \rightarrow t_{i+1} | \mathbf{z}_i, \mathbf{z}_{i+1})}_{\approx \int_{t_{i+1}}^{t_i} \mathbf{d}(\mathbf{z}_\tau, \tau) d\tau} \\
 & + \underbrace{\Delta(t_i \rightarrow t_{i-1} | \mathbf{z}_i, \mathbf{z}_{i+1})}_{\approx \int_{t_i}^{t_{i-1}} \mathbf{d}(\mathbf{z}_\tau, \tau) d\tau}, \tag{41}
 \end{aligned}$$

where $\gamma_1, \gamma_2 \in [0, 1]$. It is expected that with both $\mathbf{z}_i$ and $\mathbf{z}_{i+1}$, the two integration approximations $\Delta(t_i \rightarrow t_{i+1} | \mathbf{z}_i, \mathbf{z}_{i+1})$ and $\Delta(t_{i+1} \rightarrow t_{i+2} | \mathbf{z}_i, \mathbf{z}_{i+1})$ would become more accurate than the ones based on $\mathbf{z}_i$ or $\mathbf{z}_{i+1}$ alone. It is immediate that (41) is also reversible. That is, $\mathbf{z}_{i+2}$ can be recovered from $(\mathbf{z}_{i+1}, \mathbf{z}_i, \mathbf{z}_{i-1})$ by simple linear algebra.

We note that the update expression (41) might be more accurate in computing $\mathbf{z}_{i-1}$ than (24). However, it would also require additional memory to store the three diffusion states $\{\mathbf{z}_j\}_{j=i}^{i+2}$ at timestep i , which is not always desirable. See Subsection 5.3 for the discussion of the limitations of BDIA.

B Proof of Theorem 1

Proof. As we mentioned earlier, our convergence analysis for BDIA-DDIM is inspired by the analysis for the linear multi-step method as provided in Theorem 6.6 of [2].

Before presenting the proof, it is noted from [22] that the DDIM update expression for computing $\mathbf{z}_t$ based on $\mathbf{z}_s$ can be interpreted as a first-order integration approximation of

$$\mathbf{z}_t = \frac{\alpha_t}{\alpha_s} \mathbf{z}_s + \alpha_t \int_{\lambda_s}^{\lambda_t} \hat{\mathbf{e}}_\theta(\mathbf{z}_\lambda, \lambda) d\lambda. \tag{42}$$

We will use (42) later on for the convergence analysis.

Now we present our argument. We first reformulate the update expression for BDIA-DDIM as below:

$$\begin{aligned}
& \mathbf{z}_{i-1} \\
&= \gamma \mathbf{z}_{i+1} - \gamma \left[\frac{\mathbf{z}_i}{a_{i+1}} - \frac{b_{i+1}}{a_{i+1}} \hat{\mathbf{e}}_{\theta}(\mathbf{z}_i, i) \right] + \left[a_i \mathbf{z}_i + b_i \hat{\mathbf{e}}_{\theta}(\mathbf{z}_i, i) \right] \\
&\stackrel{(a)}{=} \gamma \mathbf{z}_{i+1} - \gamma \left[\frac{\alpha_{i+1}}{\alpha_i} \mathbf{z}_i + \underbrace{\alpha_{i+1} h \hat{\mathbf{e}}_{\theta}(\mathbf{z}_i, i)}_{\approx \alpha_{i+1} \int_{\lambda_i}^{\lambda_{i+1}} \hat{\mathbf{e}}_{\theta}(\mathbf{z}_{\lambda}, \lambda) d\lambda} \right] \\
&\quad + \left[\frac{\alpha_{i-1}}{\alpha_i} \mathbf{z}_i - \underbrace{\alpha_{i-1} h \hat{\mathbf{e}}_{\theta}(\mathbf{z}_i, i)}_{\approx \alpha_{i-1} \int_{\lambda_i}^{\lambda_{i-1}} \hat{\mathbf{e}}_{\theta}(\mathbf{z}_{\lambda}, \lambda) d\lambda} \right] \\
&= \gamma \mathbf{z}_{i+1} + \left[\frac{\alpha_{i-1} - \gamma \alpha_{i+1}}{\alpha_i} \mathbf{z}_i - (\alpha_{i-1} + \gamma \alpha_{i+1}) h \hat{\mathbf{e}}_{\theta}(\mathbf{z}_i, i) \right], \tag{43}
\end{aligned}$$

where in step (a), we indicate that the DDIM update expressions are indeed the first order integration approximation of (42) for the two time intervals $[t_i, t_{i-1}]$ and $[t_i, t_{i+1}]$.

By applying the component-wise Taylor expansion for (42) over the two time intervals $[t_i, t_{i-1}]$ and $[t_i, t_{i+1}]$, it is not difficult to show that the ground-truth diffusion state $\mathbf{z}_{i-1}^*$ can be represented as

$$\begin{aligned}
\mathbf{z}_{i-1}^* &= \gamma \mathbf{z}_{i+1}^* + \left[\frac{\alpha_{i-1} - \gamma \alpha_{i+1}}{\alpha_i} \mathbf{z}_i^* - (\alpha_{i-1} + \gamma \alpha_{i+1}) h \hat{\mathbf{e}}_{\theta}(\mathbf{z}_i^*, i) \right] \\
&\quad + \underbrace{\frac{1}{2} \alpha_{i-1} h^2 \phi'_{\theta, \{\xi_{im}\}_{m=1}^d}}_{\text{1st term}} - \underbrace{\frac{\gamma}{2} \alpha_{i+1} h^2 \phi'_{\theta, \{\tilde{\xi}_{im}\}_{m=1}^d}}_{\text{2nd term}}, \tag{44}
\end{aligned}$$

where

$$\phi'_{\theta, \{\xi_{im}\}_{m=1}^d} = \left[\frac{1}{\lambda'_{\xi_{i1}}} \hat{\mathbf{e}}'_{\theta}(\mathbf{z}_{\xi_{i1}}^*, \xi_{i1})[1], \dots, \frac{1}{\lambda'_{\xi_{id}}} \hat{\mathbf{e}}'_{\theta}(\mathbf{z}_{\xi_{id}}^*, \xi_{id})[d] \right]^T \tag{45}$$

$$\phi'_{\theta, \{\tilde{\xi}_{im}\}_{m=1}^d} = \left[\frac{1}{\lambda'_{\tilde{\xi}_{i1}}} \hat{\mathbf{e}}'_{\theta}(\mathbf{z}_{\tilde{\xi}_{i1}}^*, \tilde{\xi}_{i1})[1], \dots, \frac{1}{\lambda'_{\tilde{\xi}_{id}}} \hat{\mathbf{e}}'_{\theta}(\mathbf{z}_{\tilde{\xi}_{id}}^*, \tilde{\xi}_{id})[d] \right]^T, \tag{46}$$

where $t_i \geq \xi_{im} \geq t_{i-1}$ and $t_{i+1} \geq \tilde{\xi}_{im} \geq t_i$, and $\hat{\mathbf{e}}'_{\theta}(\mathbf{z}_{\xi_{ij}}^*, \xi_{ij})$ and $\lambda'_{\xi_{ij}}$ denote the differentials of $\hat{\mathbf{e}}_{\theta}(\mathbf{z}_{\xi_{ij}}^*, \xi_{ij})$ and $\lambda_{\xi_{ij}}$ at $t = \xi_{ij}$, respectively. The 1st and 2nd terms represent the high order errors of the integration approximations over $[t_i, t_{i-1}]$ and $[t_i, t_{i+1}]$, respectively.

Subtraction (44) from (43) yields

$$\begin{aligned}
& \mathbf{e}_{i-1} \\
&= \gamma \mathbf{e}_{i+1} + \left[\frac{\alpha_{i-1} - \gamma \alpha_{i+1}}{\alpha_i} \mathbf{e}_i \right. \\
&\quad \left. - (\alpha_{i-1} + \gamma \alpha_{i+1}) h (\hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{z}_i^*, i) - \hat{\mathbf{e}}_{\boldsymbol{\theta}}(\mathbf{z}_i, i)) \right] \\
&\quad + \frac{1}{2} \alpha_{i-1} h^2 \phi'_{\boldsymbol{\theta}, \{\xi_{im}\}_{m=1}^d} - \frac{\gamma}{2} \alpha_{i+1} h^2 \phi'_{\boldsymbol{\theta}, \{\bar{\xi}_{im}\}_{m=1}^d}
\end{aligned} \tag{47}$$

Taking the l_1 norm on both sides of (47), applying both the triangle inequality and the Lipschitz condition (30) and (31) produces

$$\begin{aligned}
\|\mathbf{e}_{i-1}\|_1 &\leq \gamma \|\mathbf{e}_{i+1}\|_1 + \frac{\alpha_{i-1} - \gamma \alpha_{i+1}}{\alpha_i} \|\mathbf{e}_i\|_1 + (\alpha_{i-1} + \gamma \alpha_{i+1}) h K_1 \|\mathbf{e}_i\|_1 \\
&\quad + \frac{1}{2} \alpha_{i-1} h^2 K_2 d + \frac{\gamma}{2} h^2 \alpha_{i+1} K_2 d \\
&\leq \gamma \|\mathbf{e}_{i+1}\|_1 + \frac{\alpha_{i-1} - \gamma \alpha_{i+1}}{\alpha_{i+1}} \|\mathbf{e}_i\|_1 + (\alpha_0 + \gamma \alpha_2) h K_1 \|\mathbf{e}_i\|_1 \\
&\quad + \frac{\alpha_0 + \gamma \alpha_2}{2} h^2 K_2 d,
\end{aligned} \tag{48}$$

where d denotes the dimensionality of $\hat{\mathbf{e}}_{\boldsymbol{\theta}}$, and the second inequality is derived by using the fact that α_t is non-increasing as t increases.

Next we derive an upper-bound for $\|\mathbf{e}_{i-1}\|_1$. To do so, we introduce the following error bounding function

$$f_i = \max_{i \leq k \leq N} \|\mathbf{e}_k\|_1. \tag{49}$$

By inspection of (32) and (49), it is immediate that

$$f_{N-1} = \eta(h). \tag{50}$$

Upon introducing f_i in (49), the inequality (48) can be further relaxed to be

$$\begin{aligned}
\|\mathbf{e}_{i-1}\|_1 &\leq \gamma f_i + \frac{\alpha_{i-1} - \gamma \alpha_{i+1}}{\alpha_{i+1}} f_i + (\alpha_0 + \gamma \alpha_2) h K_1 f_i \\
&\quad + \frac{\alpha_0 + \gamma \alpha_2}{2} h^2 K_2 d \\
&= \left(1 + \frac{\alpha_{i-1} - \alpha_{i+1}}{\alpha_{i+1}} + (\alpha_0 + \gamma \alpha_2) h K_1 \right) f_i + \frac{\alpha_0 + \gamma \alpha_2}{2} h^2 K_2 d
\end{aligned} \tag{51}$$

$$\begin{aligned}
&= \left(1 + \frac{\alpha_{i-1}/\sigma_{i+1} - \gamma\alpha_{i+1}/\sigma_{i+1}}{\alpha_{i+1}/\sigma_{i+1}} + (\alpha_0 + \gamma\alpha_2)hK_1 \right) f_i + \frac{\alpha_0 + \gamma\alpha_2}{2} h^2 K_2 d \\
&\stackrel{(a)}{\leq} \left(1 + \frac{\alpha_{i-1}/\sigma_{i-1} - \alpha_{i+1}/\sigma_{i+1}}{\alpha_{i+1}/\sigma_{i+1}} + (\alpha_0 + \gamma\alpha_2)hK_1 \right) f_i + \frac{\alpha_0 + \gamma\alpha_2}{2} h^2 K_2 d \\
&= \left(1 + \frac{1/\lambda_{i-1} - 1/\lambda_{i+1}}{1/\lambda_{i+1}} + (\alpha_0 + \gamma\alpha_2)hK_1 \right) f_i + \frac{\alpha_0 + \gamma\alpha_2}{2} h^2 K_2 d \\
&= \left(1 + \frac{\lambda_{i+1} - \lambda_{i-1}}{\lambda_{i-1}} + (\alpha_0 + \gamma\alpha_2)hK_1 \right) f_i + \frac{\alpha_0 + \gamma\alpha_2}{2} h^2 K_2 d \\
&\stackrel{(b)}{\leq} \left(1 + \frac{2h}{\lambda_0} + (\alpha_0 + \gamma\alpha_2)hK_1 \right) f_i + \frac{\alpha_0 + \gamma\alpha_2}{2} h^2 K_2 d \\
&= (1 + \beta h) f_i + \frac{\alpha_0 + \gamma\alpha_2}{2} h^2 K_2 d \tag{52}
\end{aligned}$$

where $\beta = \frac{2}{\lambda_0} + (\alpha_0 + \gamma\alpha_2)K_1$. Step (a) in (52) is derived by using the fact that σ_t is non-decreasing as t increases. Step (b) follows from the fact that λ_t is increasing as t increases.

Since the RHS of (52) is greater or equal to f_i , the inequality (52) can be reformulated as

$$f_{i-1} \leq (1 + \beta h) f_i + \frac{\alpha_0 + \gamma\alpha_2}{2} h^2 K_2 d. \tag{53}$$

Applying (53) recursively yields

$$\begin{aligned}
f_{i-1} &\leq (1 + \beta h)^{N-i} f_{N-1} + \frac{\alpha_0 + \gamma\alpha_2}{2} h^2 K_2 d \left(\sum_{j=i}^{N-1} (1 + \beta h)^{N-1-j} \right) \\
&= (1 + \beta h)^{N-i} f_{N-1} + \frac{\alpha_0 + \gamma\alpha_2}{2} h^2 K_2 d \left(\sum_{j=0}^{N-1-i} (1 + \beta h)^j \right) \\
&= (1 + \beta h)^{N-i} f_{N-1} + \frac{\alpha_0 + \gamma\alpha_2}{2} h^2 K_2 d \frac{1 - (1 + \beta h)^{N-i}}{\beta h} \\
&\stackrel{(a)}{\leq} e^{\beta h(N-i)} f_{N-1} + \frac{\alpha_0 + \gamma\alpha_2}{2} h K_2 d \frac{1 - e^{\beta h(N-i)}}{\beta} \\
&\leq e^{\beta(\lambda_N - \lambda_0)} f_{N-1} + \frac{\alpha_0 + \gamma\alpha_2}{2\beta} K_2 d (1 - e^{\beta(\lambda_N - \lambda_0)}) h \\
&\stackrel{(b)}{=} e^{\beta(\lambda_N - \lambda_0)} \eta(h) + \frac{\alpha_0 + \gamma\alpha_2}{2\beta} K_2 d (1 - e^{\beta(\lambda_N - \lambda_0)}) h, \tag{54}
\end{aligned}$$

where step (a) uses the inequality that $\log(1 + x) \leq x$ for all $x > 0$ and step (b) uses (50). (33) follows directly from (54) by using the fact that $\|e_j\|_1 \leq f_j$. The proof is complete.

C BDIA for EDM sampling procedure

C.1 Design of BDIA-EDM

In this section, we explain how to apply our new technique BDIA to the EDM sampling procedure [16]. We emphasize that the new method BDIA-EDM is designed to improve the sampling quality instead of enabling inversion of EDM.

In brief, the recent work [16] reparameterizes the forward diffusion process (1) to be

$$q_{t|0}(\mathbf{z}_t|\mathbf{x}) = \mathcal{N}(\mathbf{z}_t|\alpha_t\mathbf{x}, \alpha_t^2\tilde{\sigma}_t^2\mathbf{I}), \quad (55)$$

where σ_t of (1) is represented as $\sigma_t = \alpha_t\tilde{\sigma}_t$. The EDM sampling procedure is then designed in [16] to solve the corresponding ODE

$$d\mathbf{z} = \underbrace{\left[f(t)\mathbf{z}_t - \frac{1}{2}g^2(t)\nabla_{\mathbf{z}} \log q(\mathbf{z}_t; \alpha_t, \sigma_t) \right]}_{d(\mathbf{z}_t, t)} dt, \quad (56)$$

using an improved Euler method. In particular, the diffusion state $\mathbf{z}_{i-1}$ at timestep t_{i-1} given $\mathbf{z}_i$ is computed as

$$\tilde{\mathbf{z}}_{i-1} = \mathbf{z}_i + (t_{i-1} - t_i)\mathbf{d}_i \quad (57)$$

$$\mathbf{z}_{i-1} = \mathbf{z}_i + (t_{i-1} - t_i)\left(\frac{1}{2}\mathbf{d}_i + \frac{1}{2}\mathbf{d}'_{i-1|i}\right) \quad (58)$$

where $\mathbf{d}_i = d(\mathbf{z}_i, t_i)$ and $\mathbf{d}'_{i-1|i} = d(\tilde{\mathbf{z}}_{i-1}, t_{i-1})$. $\tilde{\mathbf{z}}_{i-1}$ is the intermediate estimate of the diffusion state $\mathbf{z}$ at time t_{i-1} . $\mathbf{z}_{i-1}$ is computed by utilizing the average of the two gradients $\mathbf{d}_i$ and $\mathbf{d}'_{i-1|i}$.

Next we consider refining the estimate for $\mathbf{z}_i$ (highlighted in blue in (57)-(58)) via BDIA when computing $\mathbf{z}_{i-1}$. We observe that in the updates of EDM, both $\mathbf{d}'_{i|i+1} = d(\tilde{\mathbf{z}}_i, t_i)$ and $\mathbf{d}_i = d(\mathbf{z}_i, t_i)$ are evaluated at the same timepoint t_i . Specifically, $\mathbf{d}'_{i|i+1}$ is for approximating $\int_{t_{i+1}}^{t_i} d(\mathbf{z}_\tau, \tau) d\tau$ over the previous timeslot $[t_{i+1}, t_i]$ while $\mathbf{d}_i$ is for approximating $\int_{t_i}^{t_{i-1}} d(\mathbf{z}_\tau, \tau) d\tau$ over the current timeslot $[t_i, t_{i-1}]$. It is thus natural to reuse $\mathbf{d}_i$ for better approximating the previous integration $\int_{t_{i+1}}^{t_i} d(\mathbf{z}_\tau, \tau) d\tau$. Similarly to BDIA-DDIM, the new estimate $\hat{\mathbf{z}}_i$ is computed as

$$\begin{aligned} \hat{\mathbf{z}}_i &= \mathbf{z}_{i+1} + (1 - \gamma)(\mathbf{z}_i - \mathbf{z}_{i+1}) \\ &\quad + \gamma(t_i - t_{i+1})\left(\frac{1}{2}\mathbf{d}_{i+1} + \frac{1}{2}\mathbf{d}_i\right), \end{aligned} \quad (59)$$

where $\gamma \in [0, 1]$. Once $\hat{\mathbf{z}}_i$ is obtained, it is then employed to replace $\mathbf{z}_i$ in blue in (57)-(58). We summarize the update procedure of BDIA-EDM in Alg. 4.

From a high-level point of view, the expression $(t_i - t_{i+1})\left(\frac{1}{2}\mathbf{d}_{i+1} + \frac{1}{2}\mathbf{d}_i\right)$ in (59) can also be interpreted as a backward integration approximation for $\int_{t_{i+1}}^{t_i} d(\mathbf{z}, t) dt$. This is because the gradient $\mathbf{d}_i$ is obtained by feeding $\mathbf{z}_i$ to the DNN model for estimating the next diffusion state $\mathbf{z}_{i-1}$.

Remark 2. Inspired by the design of BDIA-EDM, we have also proposed BDIA-DPM-Solver++ as an extension of DPM-Solver++. See Appendix E for details.

Algorithm 3 BDIA-EDM for improving sampling quality

```

1: Input: number of time steps  $N$ ,  $\gamma \in [0, 1]$ 
2: Sample  $\mathbf{z}_N \sim \mathcal{N}(\mathbf{0}, \alpha_{t_N}^2 \tilde{\sigma}_{t_N}^2 \mathbf{I})$ 
3: for  $i \in \{N, N-1, \dots, 1\}$  do
4:    $\mathbf{d}_i = \mathbf{d}(\mathbf{z}_i, t_i)$ 
5:   if  $i < N$  and  $\sigma_{t_{i-1}} \neq 0$  then
6:      $\hat{\mathbf{z}}_i = \mathbf{z}_{i+1} + (1 - \gamma)(\mathbf{z}_i - \mathbf{z}_{i+1}) + \gamma(t_i - t_{i+1})$ 
7:      $\cdot (\frac{1}{2}\mathbf{d}_{i+1} + \frac{1}{2}\mathbf{d}_i)$  [averaged integration approx.]
8:   else
9:      $\hat{\mathbf{z}}_i = \mathbf{z}_i$ 
10:  end if
11:   $\tilde{\mathbf{z}}_{i-1} \leftarrow \hat{\mathbf{z}}_i + (t_{i-1} - t_i)\mathbf{d}_i$ 
12:  if  $\sigma_{t_{i-1}} \neq 0$  then
13:     $\mathbf{d}'_{i-1|i} = \mathbf{d}(\tilde{\mathbf{z}}_{i-1}, t_{i-1})$ 
14:     $\mathbf{z}_{i-1} \leftarrow \hat{\mathbf{z}}_i + (t_{i-1} - t_i) (\frac{1}{2}\mathbf{d}_i + \frac{1}{2}\mathbf{d}'_{i-1|i})$ 
15:  end if
16: Output:  $\mathbf{z}_0$ 

```

C.2 Experimental evaluation of BDIA-EDM

Table 3: FID comparison of EDM and BDIA-EDM. The reason for testing smaller NFEs for CIFAR10 than those for other datasets is because in [16], the optimal NFE for CIFAR10 is smaller than others.

	CIFAR10	FFHQ	AFHQV2	ImageNet64
NFEs	35	39	39	39
EDM	1.85	2.64	2.08	2.51
BDIA-EDM	1.79	2.54	2.02	2.38
NFEs	43	47	47	47
EDM	1.85	2.55	2.06	2.44
BDIA-EDM	1.80	2.46	2.01	2.35

D Coupled Bidirectional Integration Approximation (CBDIA) as an Extension of EDICT

In this section, we introduce the concept of bidirectional integration approximation into EDICT. The new sampling method is referred to as *coupled bidirectional integration approximation (CBDIA)*. Similarly to EDICT, CBDIA also needs to evaluate the neural network model $\hat{\epsilon}_\theta$ two times in general per timestep. We will show that CBDIA under a special parameter setup reduces to BDIA with $\gamma = 1$. See Appendix I for a performance comparison of CBDIA and EDICT.

In the following, we first present the update expressions of CBDIA. We then show that CBDIA with the setup $(\gamma_1, \gamma_2) = (0, 1)$ reduces to BDIA with $\gamma = 1$. After that, we study CBDIA with the special setup $(\gamma_1, \gamma_2) = (1, 0)$. Finally, we present the update expressions for CBDIA-DDIM.

D.1 Update expressions of CBDIA

In this subsection, we present the sampling update-expressions of CBDIA. Suppose at timestep t_i , we have a pair of diffusion states $(\mathbf{z}_i, \mathbf{y}_i)$. The pair of diffusion states $(\mathbf{z}_{i-1}, \mathbf{y}_{i-1})$ at timestep t_{i-1} are computed to be

$$\mathbf{w}_{i-1} = \mathbf{z}_i + \overbrace{\Delta(t_i \rightarrow t_{i-1} | \mathbf{y}_i)}^{\approx \int_{t_i}^{t_{i-1}} \mathbf{d}(\mathbf{z}, t) dt} \quad (60)$$

$$\mathbf{v}_{i-1} = \mathbf{y}_i - \underbrace{\Delta(t_{i-1} \rightarrow t_i | \mathbf{w}_{i-1})}_{\approx - \int_{t_i}^{t_{i-1}} \mathbf{d}(\mathbf{z}, t) dt} \quad (61)$$

$$\mathbf{z}_{i-1} = \gamma_1 \mathbf{w}_{i-1} + (1 - \gamma_1) \mathbf{v}_{i-1} \quad (62)$$

$$\mathbf{y}_{i-1} = \gamma_2 \mathbf{w}_{i-1} + (1 - \gamma_2) \mathbf{v}_{i-1}, \quad (63)$$

where $\gamma_1, \gamma_2 \in [0, 1]$ and $\gamma_1 \neq \gamma_2$. It is seen from the above equations that the computation of $\mathbf{w}_{i-1}$ involves forward integration approximation $\Delta(t_i \rightarrow t_{i-1} | \mathbf{y}_i)$ while $\mathbf{v}_{i-1}$ is computed by using the backward integration approximation $\Delta(t_{i-1} \rightarrow t_i | \mathbf{w}_{i-1})$.

Now we show that (60)-(63) are also invertable. Suppose we obtain $(\mathbf{z}_{i-1}, \mathbf{y}_{i-1})$ at timestep t_{i-1} . The pair of diffusion states $(\mathbf{z}_i, \mathbf{y}_i)$ at timestep t_i can be easily computed to be

$$\mathbf{v}_{i-1} = \frac{\gamma_1 \mathbf{y}_{i-1} - \gamma_2 \mathbf{z}_{i-1}}{\gamma_1 - \gamma_2} \quad (64)$$

$$\mathbf{w}_{i-1} = \frac{(1 - \gamma_1) \mathbf{y}_{i-1} - (1 - \gamma_2) \mathbf{z}_{i-1}}{\gamma_2 - \gamma_1} \quad (65)$$

$$\mathbf{y}_i = \mathbf{v}_{i-1} + \Delta(t_{i-1} \rightarrow t_i | \mathbf{w}_{i-1}) \quad (66)$$

$$\mathbf{z}_i = \mathbf{w}_{i-1} - \Delta(t_i \rightarrow t_{i-1} | \mathbf{y}_i). \quad (67)$$

D.2 CBDIA including BDIA with $\gamma = 1$ as a special case

In this subsection, we show that CBDIA with $(\gamma_1, \gamma_2) = (0, 1)$ reduces to BDIA with $\gamma = 1$. It is straightforward that (60)-(63) under $(\gamma_1, \gamma_2) = (0, 1)$ take a special form of

$$\mathbf{y}_{i-1} = \mathbf{z}_i + \Delta(t_i \rightarrow t_{i-1} | \mathbf{y}_i) \quad (68)$$

$$\mathbf{z}_{i-1} = \mathbf{y}_i - \Delta(t_{i-1} \rightarrow t_i | \mathbf{y}_{i-1}), \quad (69)$$

which, under the assumption of $\mathbf{z}_N = \mathbf{y}_N$, can be reformulated as

$$\mathbf{y}_{N-1} = \mathbf{y}_N + \Delta(t_N \rightarrow t_{N-1} | \mathbf{y}_N) \quad (70)$$

$$\mathbf{y}_{i-1} = \mathbf{y}_{i+1} - \Delta(t_i \rightarrow t_{i+1} | \mathbf{y}_i) + \Delta(t_i \rightarrow t_{i-1} | \mathbf{y}_i) \quad i < N-1 \quad (71)$$

$$\mathbf{z}_{i-1} = \mathbf{y}_i - \Delta(t_{i-1} \rightarrow t_i | \mathbf{y}_{i-1}) \quad i < N. \quad (72)$$

One can easily conclude from (71) that the update expression for $\mathbf{y}_{i-1}$ is identical to that of BDIA with $\gamma = 1$. It is not difficult to derive the update expressions for $(\mathbf{y}_i, \mathbf{z}_i)$ in terms of the historical integration approximations. We summarize the results in a lemma below:

Lemma 1. *Suppose $\mathbf{z}_N = \mathbf{y}_N$, $\{(\mathbf{y}_i, \mathbf{z}_i) | i < N\}$ under the update procedure of (70)-(72) can be represented in terms of $\{\Delta(t_j \rightarrow t_{j-1} | \mathbf{y}_j), \Delta(t_{j-1} \rightarrow t_j | \mathbf{y}_{j-1})\}_{j=N}^{j=i-1}$ as*

$$\begin{aligned} \mathbf{y}_i &= \mathbf{y}_N + \Delta(t_N - t_{N-1} | \mathbf{y}_N) \bmod (N-i, 2) \\ &+ \sum_{j=i+1}^{N-1} (-\Delta(t_j \rightarrow t_{j+1} | \mathbf{y}_j) + \Delta(t_j \rightarrow t_{j-1} | \mathbf{y}_j)) \bmod (j-i, 2) \end{aligned} \quad (73)$$

$$\begin{aligned} \mathbf{z}_i &= \mathbf{y}_N + \Delta(t_N - t_{N-1} | \mathbf{y}_N) \bmod (N-i-1, 2) \\ &+ \sum_{j=i+2}^{N-1} (-\Delta(t_j \rightarrow t_{j+1} | \mathbf{y}_j) + \Delta(t_j \rightarrow t_{j-1} | \mathbf{y}_j)) \bmod (j-i-1, 2) \\ &- \Delta(t_i \rightarrow t_{i+1} | \mathbf{y}_i), \end{aligned} \quad (74)$$

where the directions of integration approximations for $\mathbf{z}_i$ are opposite of those for $\mathbf{y}_i$.

The results in Lemma 1 show that CBDIA with $(\gamma_1, \gamma_2) = (0, 1)$ indeed reduces to BDIA with $\gamma = 1$.

D.3 CBDIA with $(\gamma_1, \gamma_2) = (1, 0)$

In this subsection, we consider the special case of CBDIA with $(\gamma_1, \gamma_2) = (1, 0)$. It is immediate that (60)-(63) under the setup $(\gamma_1, \gamma_2) = (1, 0)$ can be simplified to be

$$\mathbf{z}_{i-1} = \mathbf{z}_i + \Delta(t_i \rightarrow t_{i-1} | \mathbf{y}_i) \quad (75)$$

$$\mathbf{y}_{i-1} = \mathbf{y}_i - \Delta(t_{i-1} \rightarrow t_i | \mathbf{z}_{i-1}). \quad (76)$$

One can easily represent $(\mathbf{y}_i, \mathbf{z}_i)$ in terms of the historical integration approximations. We summarize the results in a lemma below:

Lemma 2. *Suppose $\mathbf{z}_N = \mathbf{y}_N$, $\{(\mathbf{y}_i, \mathbf{z}_i) | i < N\}$ under the update procedure of (75)-(76) can be represented in terms of $\{\Delta(t_j \rightarrow t_{j+1} | \mathbf{z}_j)\}_{j=N-1}^i$ and $\{\Delta(t_j \rightarrow$*

$$t_{j-1}|\mathbf{y}_j)\}_{j=N}^{i+1} \text{ as}$$

$$\begin{aligned}\mathbf{y}_i &= \mathbf{y}_N - \sum_{j=i}^{N-1} \Delta(t_j \rightarrow t_{j+1}|\mathbf{z}_j) \\ \mathbf{z}_i &= \mathbf{z}_N + \sum_{j=i+1}^N \Delta(t_j \rightarrow t_{j-1}|\mathbf{y}_j).\end{aligned}$$

If we let $(\gamma_1, \gamma_2) = (\phi, 1 - \phi)$ in CBDIA, the update expressions for $(\mathbf{y}_i, \mathbf{z}_i)$ should include an average of forward and backward integration approximations per historical timeslot $[t_{j+1}, t_j]$, $j \leq i$.

D.4 CBDIA by using DDIM update procedure (CBDIA-DDIM)

In this subsection, we consider the special case that the DDIM update procedure is used to approximate $\Delta(t_i \rightarrow t_{i-1}|\mathbf{y}_i)$ and $\Delta(t_{i-1} \rightarrow t_i|\mathbf{w}_{i-1})$ in (60)-(61). It is immediate that (60)-(63) can be represented as

$$\mathbf{w}_{i-1} = \mathbf{z}_i + \alpha_{i-1} \overbrace{\left(\frac{\mathbf{y}_i - \sigma_i \hat{\epsilon}_{\boldsymbol{\theta}}(\mathbf{y}_i, i)}{\alpha_i} \right)}^{\text{forward DDIM update}} + \sigma_{i-1} \hat{\epsilon}_{\boldsymbol{\theta}}(\mathbf{y}_i, i) - \mathbf{y}_i \quad (77)$$

$$\mathbf{v}_{i-1} = \mathbf{y}_i - \left(\overbrace{\alpha_i \left(\frac{\mathbf{w}_{i-1} - \sigma_{i-1} \hat{\epsilon}_{\boldsymbol{\theta}}(\mathbf{w}_{i-1}, i-1)}{\alpha_{i-1}} \right)}^{\text{backward DDIM update}} + \sigma_i \hat{\epsilon}_{\boldsymbol{\theta}}(\mathbf{w}_{i-1}, i-1) - \mathbf{w}_{i-1} \right) \quad (78)$$

$$\mathbf{z}_{i-1} = \gamma_1 \mathbf{w}_{i-1} + (1 - \gamma_1) \mathbf{v}_{i-1}. \quad (79)$$

$$\mathbf{y}_{i-1} = \gamma_2 \mathbf{w}_{i-1} + (1 - \gamma_2) \mathbf{v}_{i-1}. \quad (80)$$

We refer to the above sampling procedure as CBDIA-DDIM. With (77)-(80), one can then easily derive the inverse update expressions.

E Design of BDIA-DPM-Solver++

The method DPM-Solver++⁹ in StableDiffusion V2 has a configurable implementation with several options. The results in Table 1 were obtained by using the multi-step 2nd-order DPM-Solver++, which is the default setup in StableDiffusion. At each timestep t_j , the pre-trained DNN model produces an estimator $\hat{\mathbf{x}}_{\boldsymbol{\theta}}(\mathbf{z}_j, \phi = P, t_j)$ of the clean-image, where P denotes the text prompt. In general, when $i < N$, the diffusion state $\mathbf{z}_{i-1}$ is computed by making use of the two most recent clean-image estimators $\{\hat{\mathbf{x}}_{\boldsymbol{\theta}}(\mathbf{z}_j, \phi = P, t_j)|j = i+1, i\}$ as well

⁹ Lu et al., DPM-Solver++: Fast Solver for Guided Sampling of Diffusion Probabilistic Models, arXiv:2211.01095.

Algorithm 4 BDIA-EDM for improving sampling quality

```

1: Input: number of time steps  $N$ ,  $\gamma \in [0, 1]$ 
2: Sample  $\mathbf{z}_N \sim \mathcal{N}(\mathbf{0}, \alpha_{t_N}^2 \tilde{\sigma}_{t_N}^2 \mathbf{I})$ 
3: for  $i \in \{N, N-1, \dots, 1\}$  do
4:    $\mathbf{d}_i = \mathbf{d}(\mathbf{z}_i, t_i)$ 
5:   if  $i < N$  and  $\sigma_{t_{i-1}} \neq 0$  then
6:      $\hat{\mathbf{z}}_i = \mathbf{z}_{i+1} + (1 - \gamma)(\mathbf{z}_i - \mathbf{z}_{i+1}) + \gamma(t_i - t_{i+1})$ 
7:      $\cdot (\frac{1}{2}\mathbf{d}_{i+1} + \frac{1}{2}\mathbf{d}_i)$  [averaged integration approx.]
8:   else
9:      $\hat{\mathbf{z}}_i = \mathbf{z}_i$ 
10:  end if
11:   $\tilde{\mathbf{z}}_{i-1} \leftarrow \hat{\mathbf{z}}_i + (t_{i-1} - t_i)\mathbf{d}_i$ 
12:  if  $\sigma_{t_{i-1}} \neq 0$  then
13:     $\mathbf{d}'_{i-1|i} = \mathbf{d}(\tilde{\mathbf{z}}_{i-1}, t_{i-1})$ 
14:     $\mathbf{z}_{i-1} \leftarrow \hat{\mathbf{z}}_i + (t_{i-1} - t_i) (\frac{1}{2}\mathbf{d}_i + \frac{1}{2}\mathbf{d}'_{i-1|i})$ 
15:  end if
16: Output:  $\mathbf{z}_0$ 

```

as the current state $\mathbf{z}_i$. For simplicity, let us denote the update expression of DPM-Solver++ for computing $\mathbf{z}_{i-1}$ at timestep t_i as

$$\mathbf{z}_{i-1} = \Gamma_{t_i \rightarrow t_{i-1}}(\mathbf{z}_i, \{\hat{\mathbf{x}}_{\theta}(\mathbf{z}_j, \phi = P, t_j) | j = i+1, i\}). \quad (81)$$

Next we present the update expressions of BDIA-DPM-Solver++, which is designed primarily for improving sampling quality rather than enabling diffusion inversion. The diffusion state $\mathbf{z}_{i-1}$ at timestep t_i is computed to be

$$\begin{aligned} \mathbf{z}_{i-1} = & \mathbf{z}_{i+1} + \overbrace{(1 - \gamma)(\mathbf{z}_i - \mathbf{z}_{i+1}) - \gamma \Delta(t_i \rightarrow t_{i+1} | \mathbf{z}_i)}^{\approx \int_{t_{i+1}}^{t_i} \mathbf{d}(\mathbf{z}, t) dt} \\ & + (\Gamma_{t_i \rightarrow t_{i-1}}(\mathbf{z}_i, \{\hat{\mathbf{x}}_{\theta}(\mathbf{z}_j, \phi = P, t_j) | j = i+1, i\}) - \mathbf{z}_i), \end{aligned} \quad (82)$$

where $\Delta(t_i \rightarrow t_{i+1} | \mathbf{z}_i)$ is the backward integration approximation over the timeslot $[t_{i+1}, t_i]$. In practice, one can employ DDIM to realize $\Delta(t_i \rightarrow t_{i+1} | \mathbf{z}_i)$.

Remark 3. In principle, our BDIA technique can also be incorporated into other high-order ODE solvers such as PLMS and DEIS by following a similar design procedure for BDIA-DPM-Solver++ presented above. We omit the details here.

F BDIA-EDM

Table 4: Tested pre-trained models in Table 1, 2 and 3

BDIA-DDIM (t2i sampling)	v2-1_512-ema-pruned.ckpt from https://huggingface.co/stabilityai/stable-diffusion-2-1-base/tree/main
BDIA-DDIM	ddim_cifar10.ckpt from https://github.com/luping-liu/PNDM ddim_celeba.ckpt from https://github.com/luping-liu/PNDM
BDIA-EDM	edm-cifar10-32x32-cond-vp.pkl from https://github.com/NVlabs/edm edm-afhqv2-64x64-uncond-vp.pkl from https://github.com/NVlabs/edm edm-ffhq-64x64-uncond-vp.pkl from https://github.com/NVlabs/edm edm-imagenet-64x64-cond-adm.pkl from https://github.com/NVlabs/edm

G Tested Pre-trained Models for BDIA-DDIM and BDIA-EDM

H Ablation Study for EDICT and BDIA-DDIM for Text-To-Image Generation

Before explaining how the hyper-parameters of EDICT and BDIA-DDIM were set to obtain Table 1, we first performed an ablation study for EDICT and BDIA-DDIM for the task of text-to-image generation. Our objective is to find out how the hyper-parameters p and γ affect the sampling performance of EDICT and BDIA-DDIM, respectively. In our experiment, we tested the setups $p \in \{0.5, 0.93\}$ for EDICT and $\gamma \in \{0.5, 1.0\}$ for BDIA-DDIM. The main reason we chose 0.93 is because in [40], the value 0.93 is the recommended setup for p in EDICT to enable effective image editing.

It is clear from Table 5 that the hyper-parameter p or γ indeed has a noticeable impact on the sampling quality of each method. Considering FID score, BDIA-DDIM performs consistently better than EDICT. As for the CLIP score, EDICT performs slightly better than BDIA-DDIM. One can also conclude from the table that the setup $p = 0.5$ for EDICT performs consistently better the setup $p = 0.93$.

Table 5: FID (the lower the better) and CLIP (the higher the better) evaluation of EDICT and BDIA-DDIM for different setups of p and γ , respectively. The considered task is text-to-image generation over StableDiffusion V2, which is the same as that of Table 1.

		BDIA-DDIM	BDIA-DDIM	EDICT	EDICT
		$\gamma = 0.5$	$\gamma = 1.0$	$p = 0.5$	$p = 0.93$
20 timesteps	FID	13.44	14.12	15.51	87.66
	CLIP	25.12	25.08	25.44	21.22
40 timesteps	FID	14.31	13.87	14.34	15.65
	CLIP	25.06	25.05	25.11	24.94

The setups for γ and p in Table 1 are as follows. The parameter γ in BDIA-DDIM was set to $\gamma = 0.5$ when the numbers of timesteps are 10 and 20. For the case of 40 timesteps, γ was set to 1.0 based on our ablation study in Table 5.

On the other hand, the parameter p in EDICT was set to $p = 0.5$ for timesteps of 10, 20, and 40.

I Performance of EDICT and CBDIA-DDIM for Conventional Sampling

In this section, we study the performance of EDICT and CBDIA-DDIM with $(\gamma_1, \gamma_2) = (1, 0)$ for conventional sampling, where the sampling procedure of CBDIA-DDIM is presented in Appendix D. We note that both EDICT and CBDIA-DDIM need to perform two NFEs per timestep in general. That is, both sampling methods have roughly the same computational complexities. We tested two pretrained models in the experiment: one for CIFAR10 and the other one for Celeba. Details of the two pretrained models are listed in Table 4. For each pretrained model, EDICT with $p \in \{0.25, 0.5, 0.93\}$ were evaluated. In addition, DDIM and BDIA-DDIM were evaluated to serve as benchmarks.

Table 6 summarizes the FID scores for different experimental setups. It is clear that CBDIA-DDIM with $(\gamma_1, \gamma_2) = (1, 0)$ performs better than EDICT for different numbers of timesteps and across different pre-trained models. On the other hand, it performs slightly worse than DDIM and BDIA-DDIM with $\gamma = 1$ in all cases. This implies that even if two NFEs are performed for each timestep of CBDIA-DDIM with $(\gamma_1, \gamma_2) = (1, 0)$ in comparison to one NFE per timestep in DDIM and BDIA-DDIM, it does not necessarily lead to better image quality. The reason that BDIA-DDIM with $\gamma = 1$ performs better than CBDIA-DDIM with $(\gamma_1, \gamma_2) = (1, 0)$ is because the former algorithm exhibits a pattern of interlaced integration-approximation directions (see Fig. 4), which is similar to the method of midpoint integration approximation.

Next we consider the performance of EDICT. The hyper-parameter p again affects its performance. For 10 and 20 timesteps, the setup $p = 0.5$ performs the best. However, when it is 40 timesteps, it is preferable to employ the setup $p = 0.93$. The results in Table 2 for EDICT are obtained by choosing the best p values from $\{0.25, 0.5, 0.93\}$.

Table 6: FID evaluation of EDICT for different setups of p . Performance of CBDIA-DDIM (see Appendix D), DDIM, and BDIA are also included to serve as benchmarks. The considered task is conventional sampling, which is the same as that of Table 2.

	timesteps	EDICT ($p = 0.25$)	EDICT ($p = 0.5$)	EDICT ($p = 0.93$)	CBDIA-DDIM ($\gamma_1, \gamma_2 = (1, 0)$)	DDIM	BDIA-DDIM ($\gamma = 1$)
Celeba	10	233.78	190.62	343.88	14.70	13.41	10.86
	20	99.97	81.44	227.29	10.28	9.45	8.86
	40	55.49	50.00	43.93	7.96	6.93	6.50
	timesteps	EDICT ($p = 0.25$)	EDICT ($p = 0.5$)	EDICT ($p = 0.93$)	CBDIA-DDIM ($\gamma_1, \gamma_2 = (1, 0)$)	DDIM	BDIA-DDIM ($\gamma = 1$)
CIFAR10	10	233.21	203.86	327.92	18.77	14.38	10.03
	20	136.90	116.72	224.44	11.83	7.51	6.29
	40	80.73	67.37	51.17	8.34	4.95	4.63

The high FID scores of EDICT might be because of the unnatural computation of $\mathbf{y}_i^{inter}$ in (10). It is clear from (9) that $\mathbf{z}_i^{inter}$ is in fact an estimate of a diffusion state at timestep t_{i-1} . However, $\mathbf{y}_i^{inter}$ is computed as

$$\mathbf{y}_i^{inter} = a_i \mathbf{y}_i + b_i \hat{\epsilon}_\theta(\mathbf{z}_i^{inter}, i).$$

The above update expression indicates that the input to the DNN model $\hat{\epsilon}_\theta$ is $(\mathbf{z}_i^{inter}, i)$ instead of $(\mathbf{z}_i^{inter}, i-1)$. In other words, $\mathbf{z}_i^{inter}$ and the timestep index i are not consistent, which may affect the accuracy of $\mathbf{y}_i^{inter}$ and consequently the sampling performance of EDICT.

J Additional Experiments on Text-to-Image Generation

Fig. 7-8 include additional results for text-to-image generation by using BDIA-DDIM and DDIM at 10 timesteps. It is seen from the two figures that BDIA-DDIM produces better image quality than DDIM.

Table 7: text-prompts in 7 and 8.

(a)	A large stuffed bear on display in a room
(b)	Many pink flowers are sitting on an old table near chairs
(c)	Words describe a cat that stares up at the camera
(d)	A double decker bus is moving along a stretch of road
(e)	A sheep with horns in a grassy green field
(f)	Vase holding some type of flower with large pink bloom
(g)	Two green birds are perched next to a gray bird
(h)	A man with a large grey and black beard and glasses and a tie
(i)	A house cat drinking water from a sink spigot
(j)	Two brown horses trotting down a green field
(k)	A teddy bear is sitting down wearing a bow.
(l)	A bunch of hay bales are in a field.

K Additional Experiments on Text-Based and ControlNet-Based Image-Editing

ControlNet-based image-editing:

Fig. 9 summarize the additional obtained results. We can conclude from these figures that BDIA-DDIM produces better images than DDIM, which is consistent with the results in Fig. 1 for text-to-image generation. The performance gain is due to the averaged forward and backward integration approximation per time-slot (see Proposition 1 in the main paper).

Text-based round-trip image-editing: Fig.10 and 11 present additional results for the task of text-based (round-trip) image editing. It is seen from the

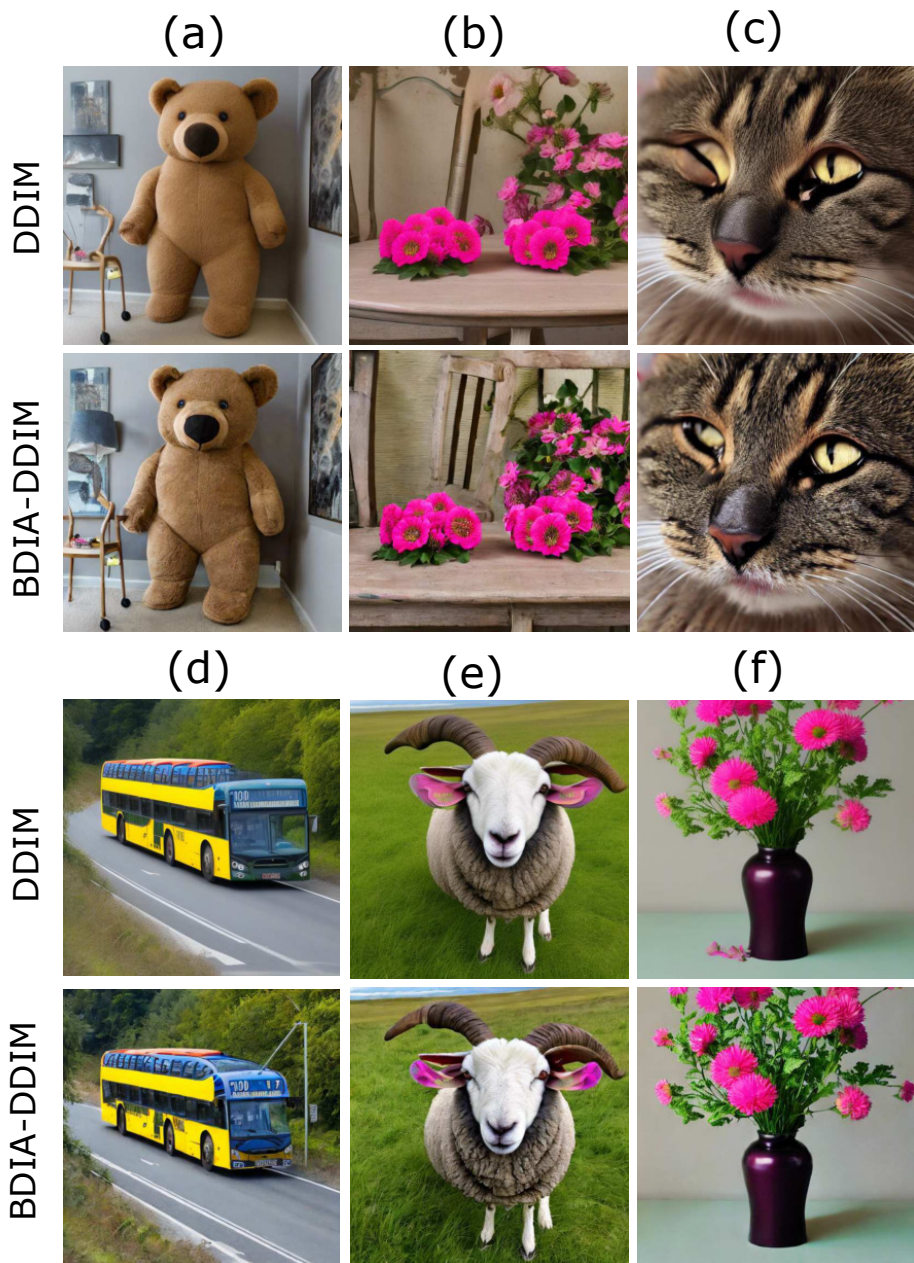


Fig. 7: Comparison of images generated by using BDIA-DDIM and DDIM with 10 timesteps over StableDiffusion V2. See Table 7 for text prompts and Table 1 for FID evaluation.

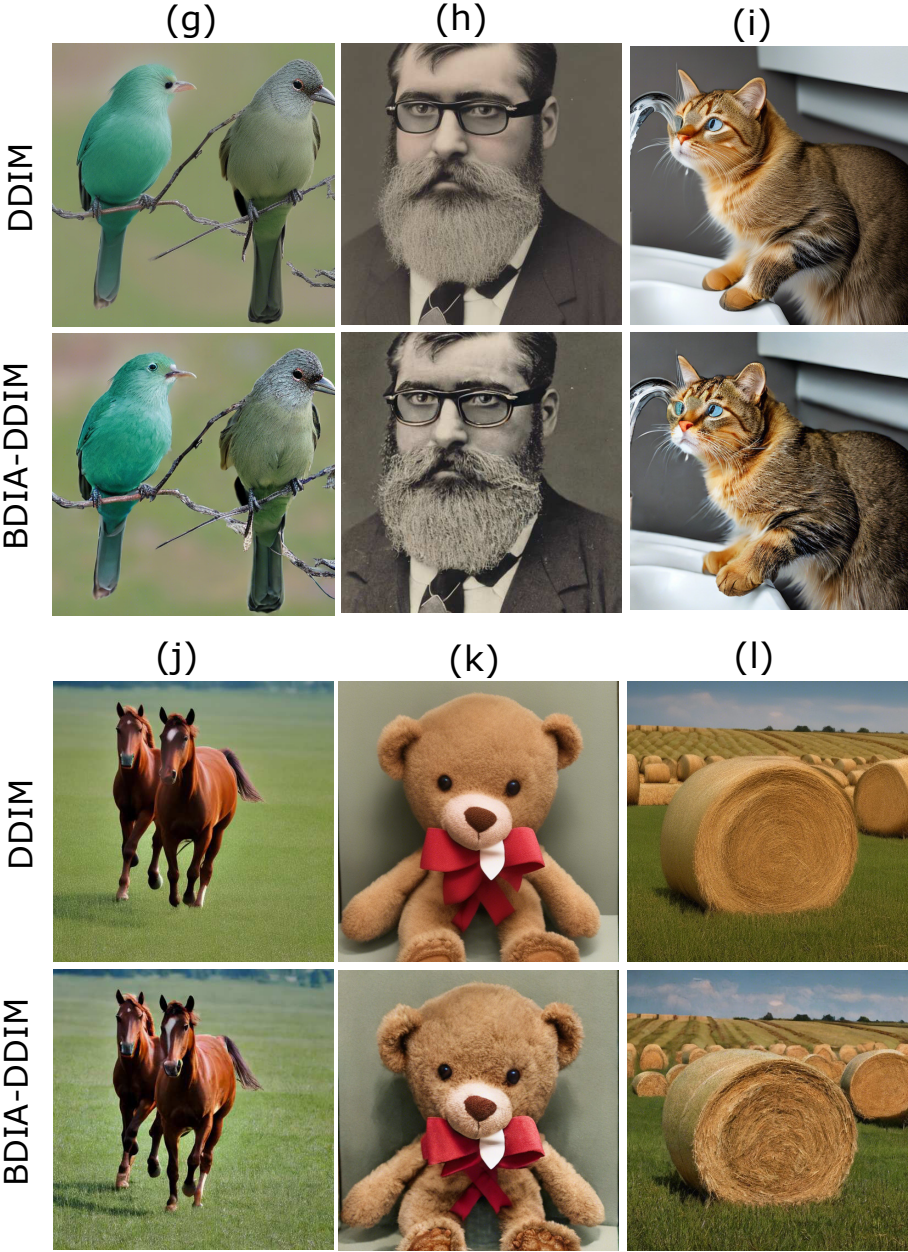


Fig. 8: Comparison of images generated by using BDIA-DDIM and DDIM with 10 timesteps over StableDiffusion V2. See Table 7 for text prompts and Table 1 for FID evaluation.

two figures that BDIA-DDIM produces comparable results to EDICT while only consuming half of NFEs needed by EDICT.

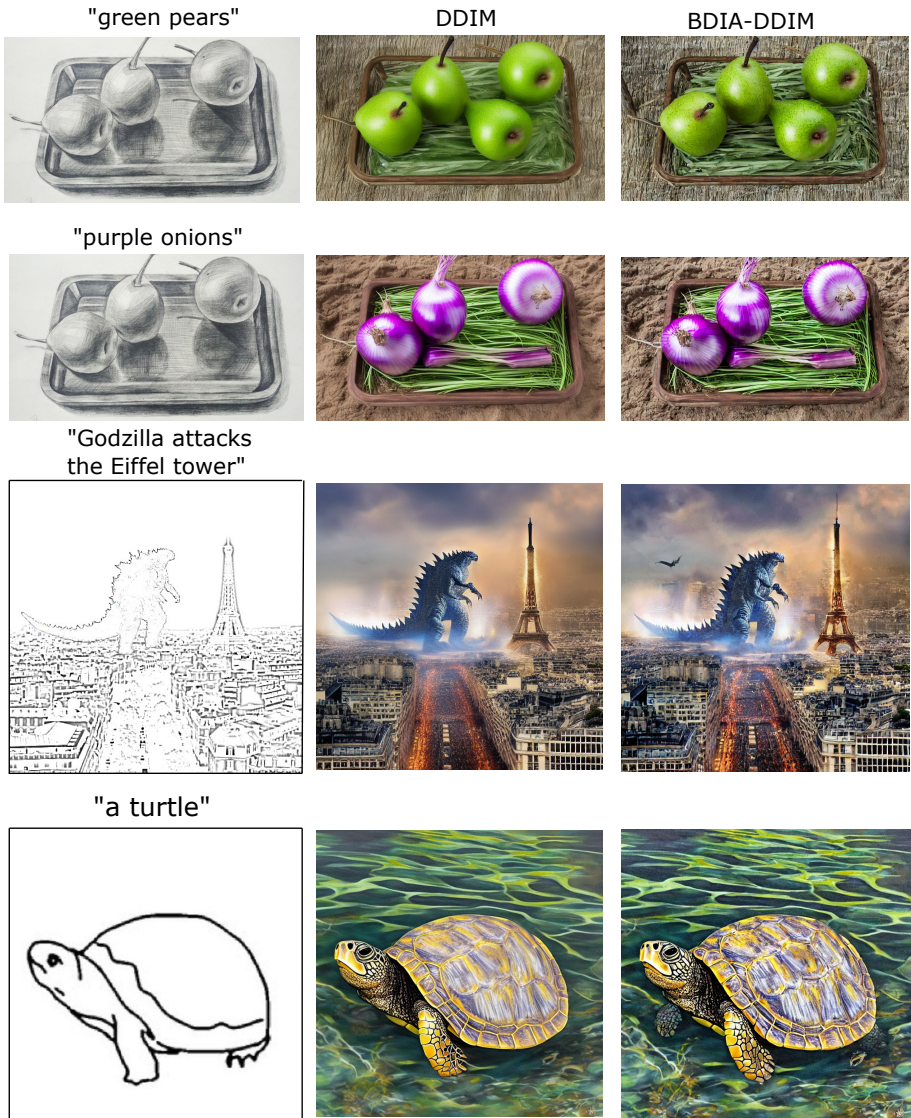


Fig. 9: ControlNet-based image editing by using BDIA-DDIM and DDIM at 10 timesteps. The hyper-parameter γ in BDIA-DDIM was set to 0.5.

One can also conclude from the figures that for some scenarios, DDIM also produces high-quality edited images. In comparison to DDIM, BDIA-DDIM provides one more degree of freedom via the parameter γ for image-editing, which makes BDIA-DDIM more flexible in practice.



Fig. 10: Round-trip image editing by using BDIA-DDIM, EDICT, and DDIM. The value $p = 0.93$ for EDICT is the recommended setup in [40]. The number of timesteps was set to 40 and the classifier guidance scale was 4.0. BDIA-DDIM only needs to perform half of NFEs needed in EDICT for each image generation.

L Experimental results of Null-text inversion

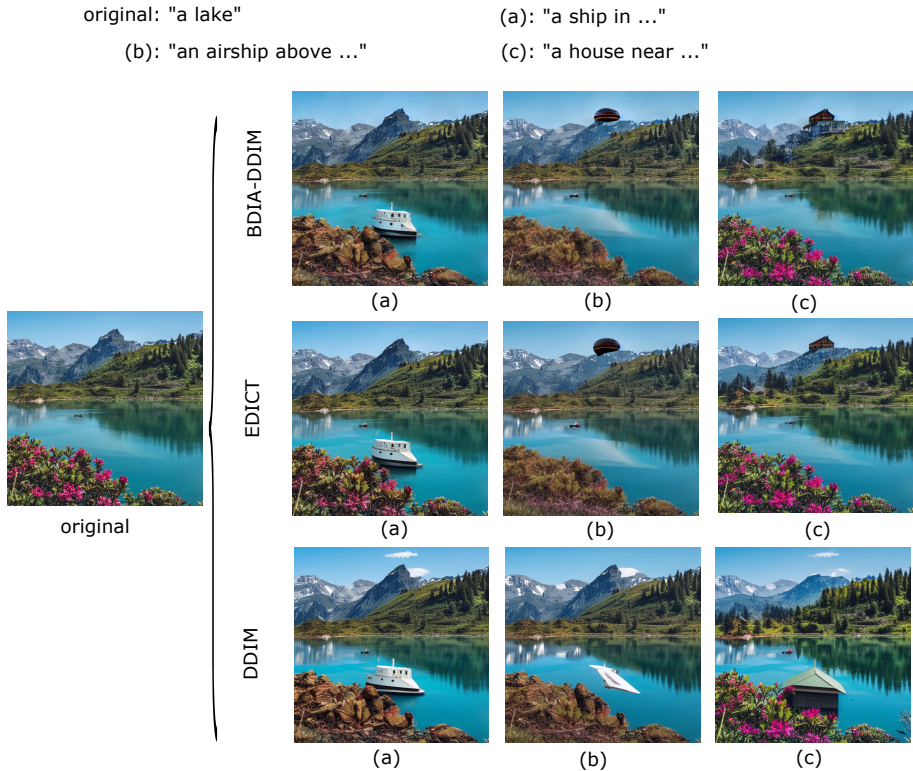


Fig. 11: Round-trip image editing by using BDIA-DDIM, EDICT, and DDIM. The hyper-parameters γ and p in BDIA-DDIM and EDICT were set to 0.98 and 0.93, respectively. The value $p = 0.93$ for EDICT is the recommended setup in [40]. The number of timesteps was set to 40 and the classifier guidance scale was 4.0. BDIA-DDIM only needs to perform half of NFEs needed in EDICT for each image generation.

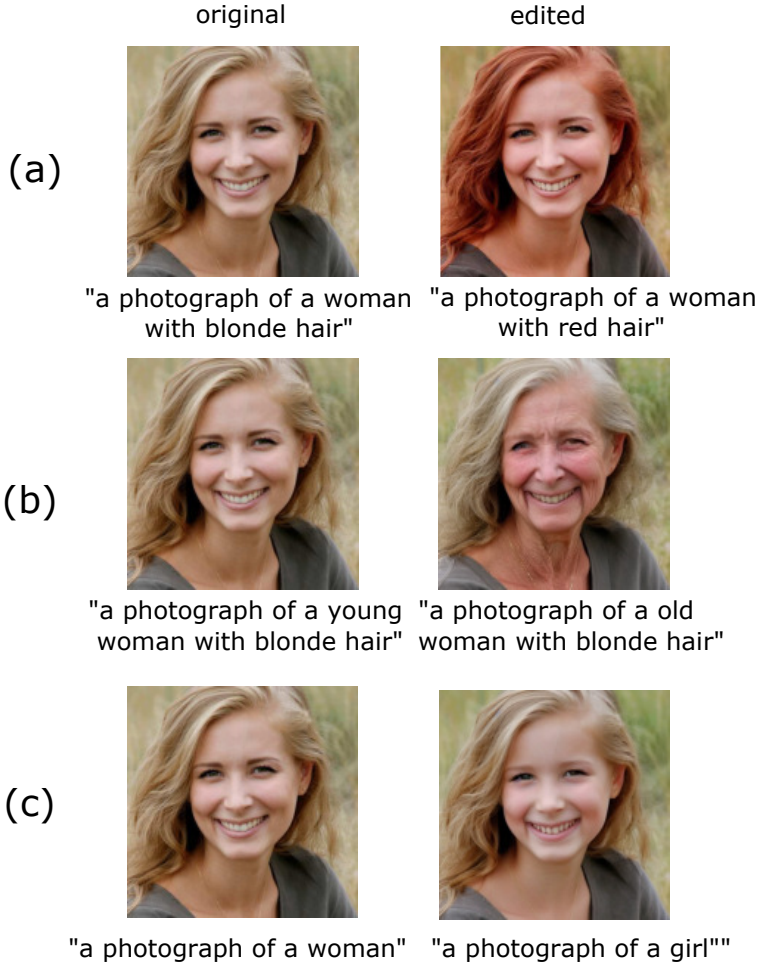


Fig. 12: Image editing via Null-text inversion. It is found that Null-text inversion is generally sensitive to the text prompts, which is the reason we choose different reference prompts in the above three scenarios: (a)-(c). On average, it took 68 seconds for generating one edited image by utilizing the Null-text inversion technique via an A100 GPU. On the other hand, the time needed for using our BDIA technique is about 18 seconds per edited image via an A100 GPU, which is therefore much faster than using the Null-text inversion.